NEURAL NETWORKS / DAY 4

Convolutional neural networks

Images as tensors, convolution, architectures, transfer learning, detection, segmentation and reconstruction

Full day · 09:00–17:00 | 7 parts · 2 laboratories | PyTorch

Today's plan

09:00	Welcome, casing-inspection case, images as tensors	Lecture
09:45	Convolution mechanics	Lecture
10:30	Break	Break
10:45	Feature maps, pooling and a compact CNN	Lecture
11:30	Lab 1: a checked CNN on CIFAR-10	Lab
12:30	Lunch	Lunch
13:30	Lab 1 debrief, architecture ideas, training and transfer learning	Lecture
14:50	Break	Break
15:05	Beyond classification; autoencoders and 1-D signals	Lecture
15:50	Lab 2: autoencoder or casing defects, then debrief and exit check	Lab

LEARNING OUTCOMES

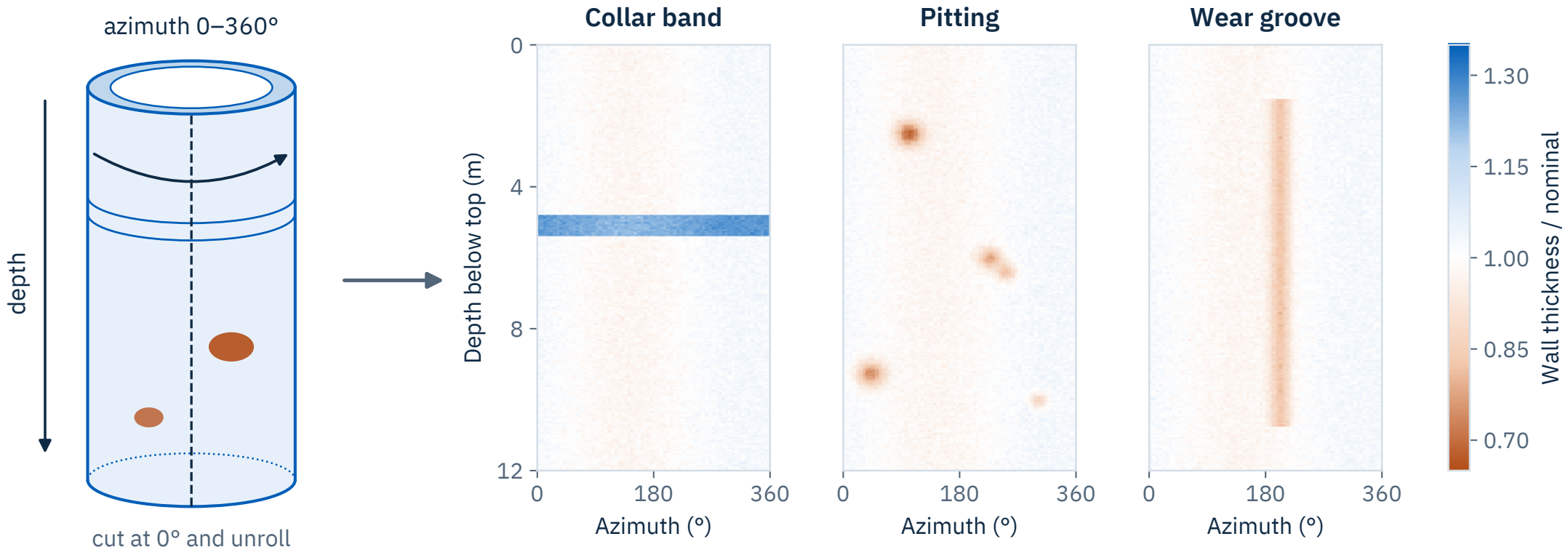
What you will be able to do

By the end of the day you will be able to:

1. Represent images and energy imaging data as tensors with the right shape and meaning.
2. Compute convolution outputs, output sizes, parameter counts and receptive fields by hand.
3. Build, shape-check and train a compact CNN in PyTorch.
4. Adapt a pretrained backbone with label-preserving augmentation and grouped evaluation.
5. Choose between classification, detection and segmentation, and test what a model uses.
6. Use convolutional autoencoders and 1-D convolutions without overstating what they show.

A casing inspection log is already an image

Ultrasonic imagers record wall thickness around the pipe at each depth: a depth × azimuth grid.



Synthetic thickness maps (seeds 4–6) with 0.05 m rows and 5° sectors, drawn to show the geometry; not tool data.

PRACTICE

4 min · Pairs

Decide what the model should return

For the casing maps on the previous slide:

1. Should the model return one label per image, per depth window, or per pixel?
2. Which neighbouring pixels would you inspect to judge whether one pixel is a pit?
3. Two windows from one well overlap by half. May one be in training and the other in test?

PART 1 OF 7

01

Images as tensors and imaging data

Know exactly which numbers the network sees.

- Greyscale grids and colour channels
- Batches in PyTorch: B × C × H × W
- Why dense layers struggle with pixels
- Seismic, borehole, core and inspection images

A greyscale image is a grid of numbers

- A scikit-learn digit is an 8 × 8 array of intensities from 0 to 16.
- Row and column indices carry meaning: neighbours are physically adjacent.
- Flattening to 64 numbers keeps the values but hides which ones are neighbours.

	0	1	2	3	4	5	6	7
0	0	0	7	15	13	1	0	0
1	0	8	13	6	15	4	0	0
2	0	2	1	13	13	0	0	0
3	0	0	2	15	11	1	0	0
4	0	0	0	1	12	12	1	0
5	0	0	0	0	1	10	8	0
6	0	0	8	4	5	14	9	0
7	0	0	7	13	13	9	0	0

rendered



8 × 8 array

64 intensities

values 0–16

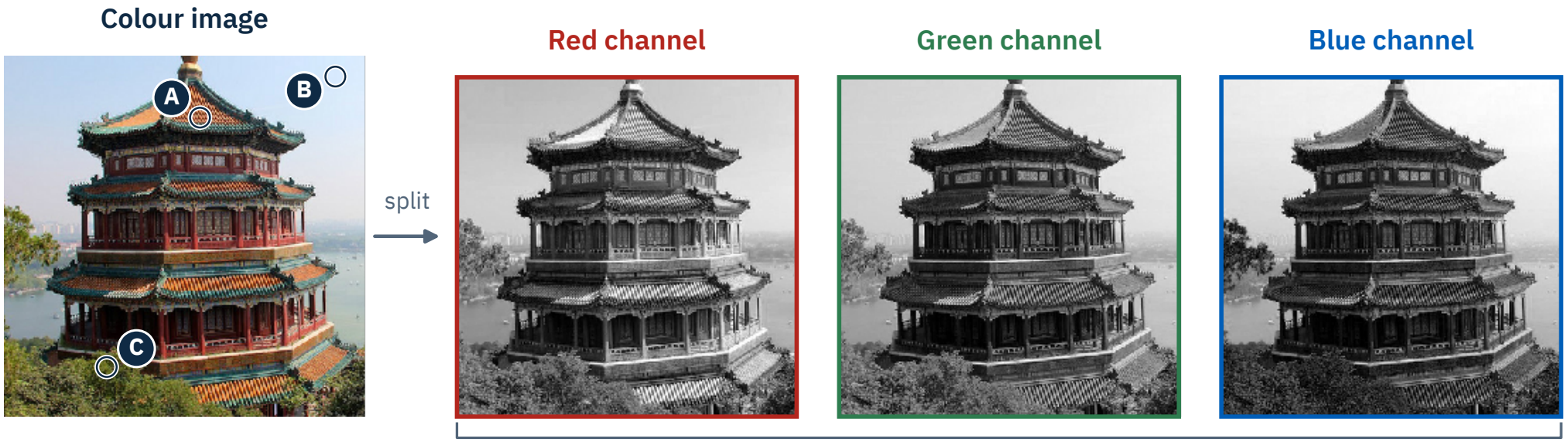
x[1, 4] = 15

row 1, column 4

scikit-learn bundled digit (index 3); exact pixel values.

A colour image stacks three channel arrays

Each pixel is a vector of three numbers; each channel on its own is a greyscale image.



A roof: (R, G, B) = (209, 121, 81)

B sky: (R, G, B) = (216, 234, 254)

C foliage: (R, G, B) = (116, 125, 42)

NumPy / PIL: H × W × C = 360 × 360 × 3, integers 0 (dark) to 255 (bright)

PyTorch: C × H × W = 3 × 360 × 360, usually rescaled to floats in [0, 1]

Crop of scikit-learn's sample image china.jpg (CC BY 2.0, danielbuechele on Flickr); exact 8-bit values.

PyTorch expects batches shaped B × C × H × W

```python
import torch
from sklearn.datasets import load_sample_image

img = load_sample_image("china.jpg")        # NumPy array: H x W x C
print(img.shape, img.dtype)

x = torch.tensor(img).permute(2, 0, 1)      # C x H x W (a copy)
x = x.float().div(255).unsqueeze(0)         # B x C x H x W, values 0-1
print(x.shape, x.dtype)
```

OUTPUT

```
(427, 640, 3) uint8
torch.Size([1, 3, 427, 640]) torch.float32
```

Keras and many image libraries default to channels-last (B × H × W × C): check before you reshape.

PRACTICE

4 min · Individual

Locality reduces the parameter burden

An RGB image has shape $32 \times 32 \times 3$.

1. How many weights connect every pixel value to 100 dense units?
2. How many weights define 16 filters of size 3×3 over the same input channels?
3. Which of the two counts changes if the image grows to 224×224?

DISCUSSION

Dense weight counts grow with every pixel

Weights (biases excluded) for 100 dense units versus 16 convolution filters of size 3 × 3:

307,200

dense layer, 32 × 32 × 3 input

15,052,800

dense layer, 224 × 224 × 3 input

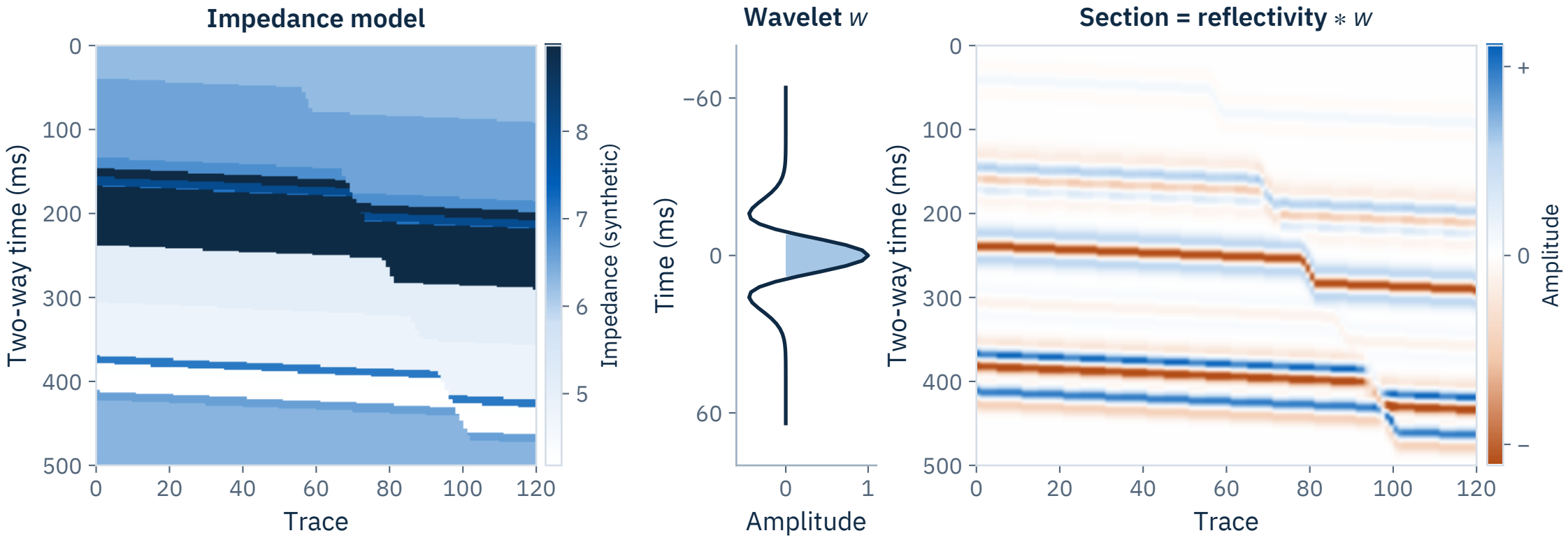
432

16 conv filters, any image size

◆ Convolution saves weights by assuming local patterns that recur; compute still grows with the pixel count.

Seismic sections are images made by convolution

Each trace is reflectivity convolved with a wavelet; faults offset the events.



Synthetic impedance model with a normal fault (seed 7), 25 Hz Ricker wavelet, 2 ms sampling; not field data.

Energy imaging comes in many modalities

Seismic

- Amplitude sections and volumes
- Faults, horizons, salt, facies

Borehole images

- Resistivity or acoustic wall images
- Fractures, bedding, breakouts

Core photographs

- Slabbed core in white or UV light
- Lithology, facies, fractures

Thin sections

- Microscope images of rock slices
- Grains, pores, minerals

Casing inspection

- Ultrasonic thickness by depth and azimuth
- Metal loss, pitting, wear

Drone and solar inspection

- RGB and thermal images of assets
- Hot spots, cracks, soiling

◆ Record pixel spacing, units, tool and acquisition run with every image: they decide augmentation and splits.

PRACTICE

4 min · Pairs

Every tensor axis has a physical meaning

A batch holds 16 casing patches of 32 depth rows × 36 azimuth sectors of 10°, one thickness channel; each row spans 0.1 m.

1. Write the PyTorch shape of the batch.
2. What depth interval and azimuth range does one patch cover?
3. Which axis wraps around, and what does that mean at the patch edges?

PART 2 OF 7

02

Convolution mechanics

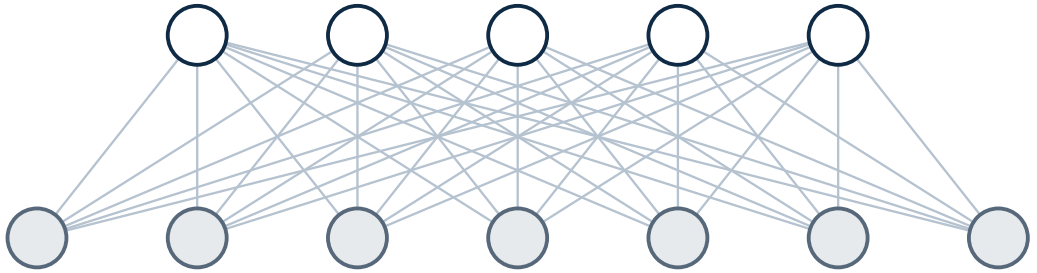
A small kernel, applied everywhere, with a few size settings.

- Weight sharing and sliding windows
- Padding, stride and dilation
- Channels and 1 × 1 convolutions
- Output sizes and parameter counts

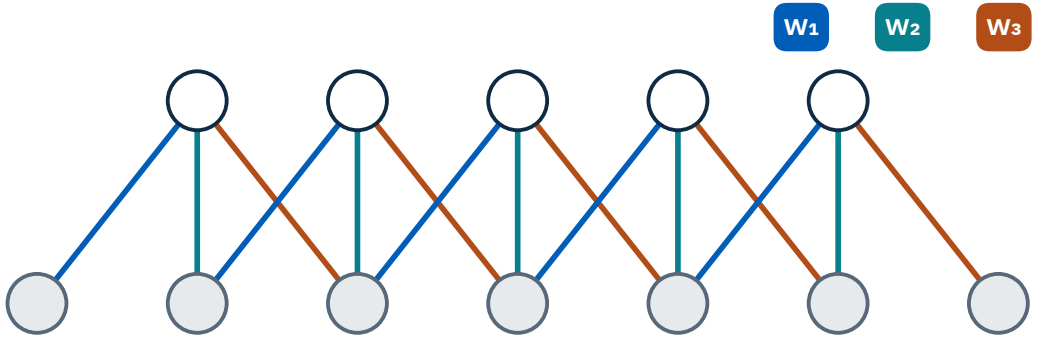
A filter applies the same rule across positions

- Each output combines a local neighbourhood of the input with learned weights.
- The same weights are reused at every position: **weight sharing**.
- A filter spans every input channel; each filter makes one output channel.
- Sharing encodes an assumption: a useful local pattern can recur anywhere.

Dense: every output sees every input — 35 weights



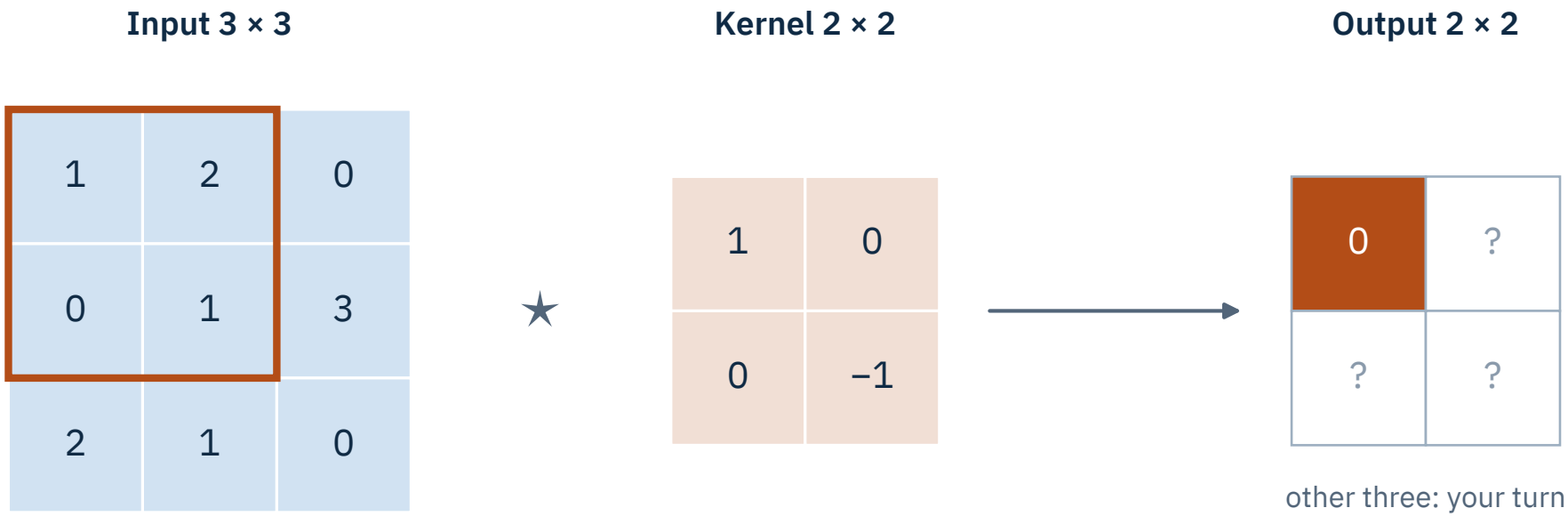
Convolution: 3 shared weights, reused — 3 weights



Schematic: seven inputs, five outputs, kernel size 3, no padding.

One output value is a weighted sum of a patch

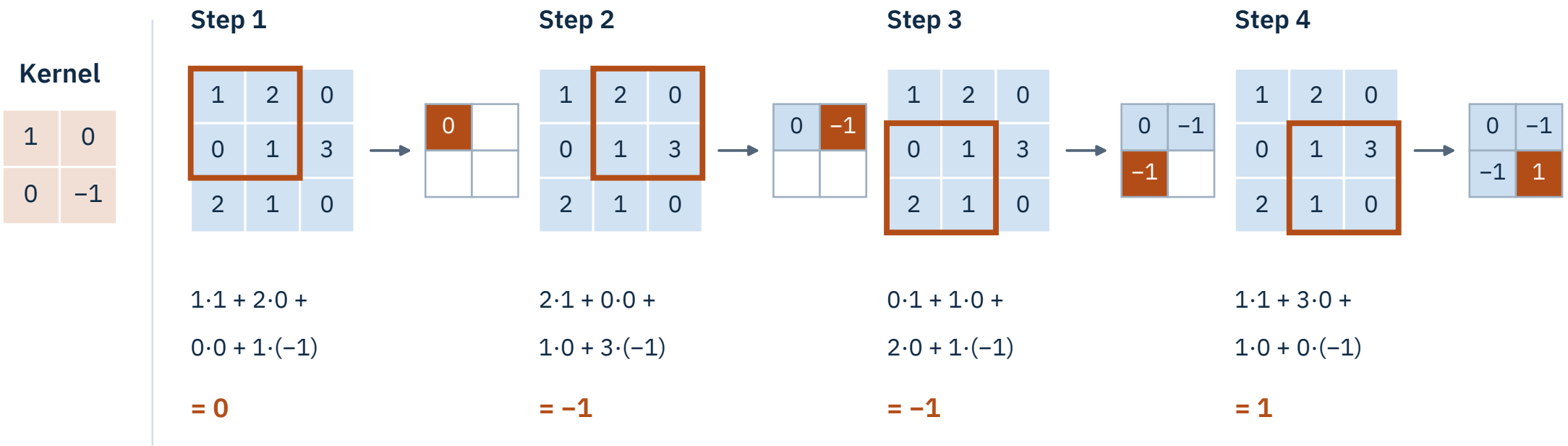
Multiply the highlighted patch by the kernel element-wise and add: 1·1 + 2·0 + 0·0 + 1·(−1) = 0.



Exact cross-correlation (scipy.signal.correlate2d) with one 2 × 2 kernel, stride 1, no padding; three outputs left for the room.

The kernel slides to fill the output map

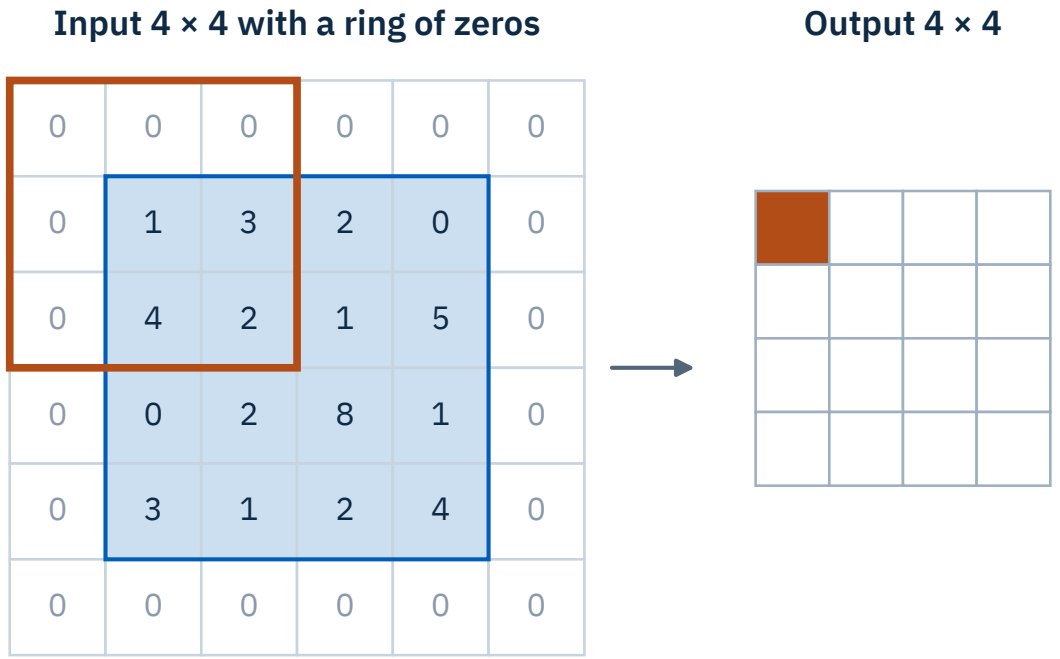
Frameworks compute cross-correlation: the kernel slides without being flipped.



Exact arithmetic on the same numbers as the previous slide; stride 1, no padding.

Zero padding keeps border pixels in play

- **Padding** adds a border so windows can centre on edge pixels.
- With K = 3, P = 1 and stride 1, the output keeps the input size.
- Border outputs mix real values with artificial zeros.



K = 3, P = 1, S = 1: $(4 + 2\cdot1 - 3)/1 + 1 = 4$

The corner window holds 5 padding zeros and 4 real values

Schematic with exact output-size arithmetic; padding width 1.

Azimuth wraps around, depth does not

```python
import torch
import torch.nn.functional as F
from torch import nn

x = torch.randn(16, 1, 32, 36)  # B, C, depth, azimuth
x = F.pad(x, (1, 1, 0, 0), mode="circular")
conv = nn.Conv2d(1, 8, kernel_size=3, padding=(1, 0))
print(x.shape, conv(x).shape)
```

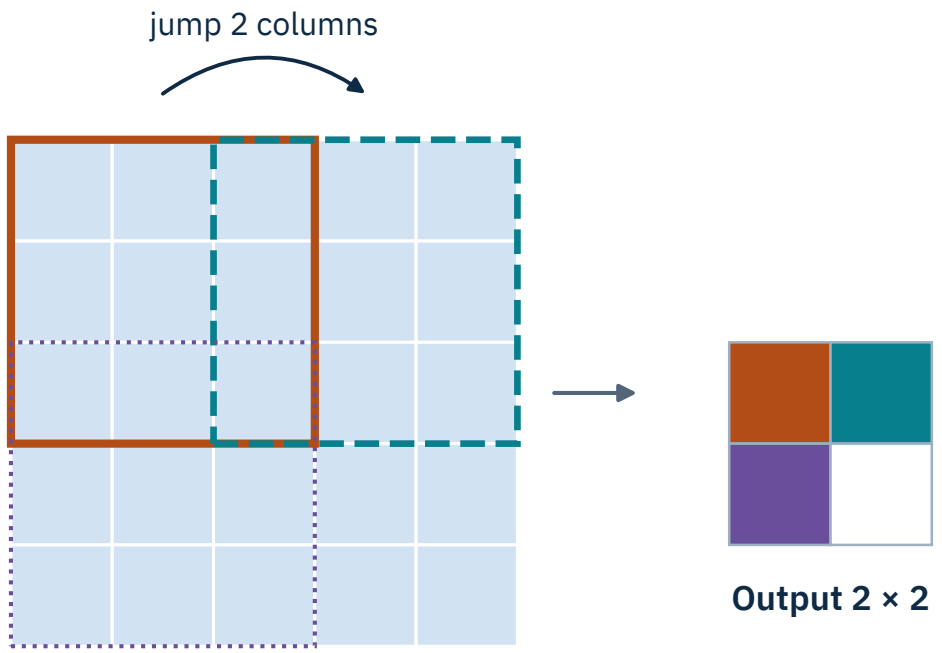
OUTPUT

```
torch.Size([16, 1, 32, 38]) torch.Size([16, 8, 32, 36])
```

- `F.pad` wraps only the last axis, azimuth.
- `padding=(1, 0)` zero-pads depth only.
- `padding_mode="circular"` would wrap depth too.

Stride skips positions and shrinks the map

- **Stride** S moves the window S pixels at each step.
- S = 2 roughly halves height and width.
- Neighbouring outputs now sit S input pixels apart.

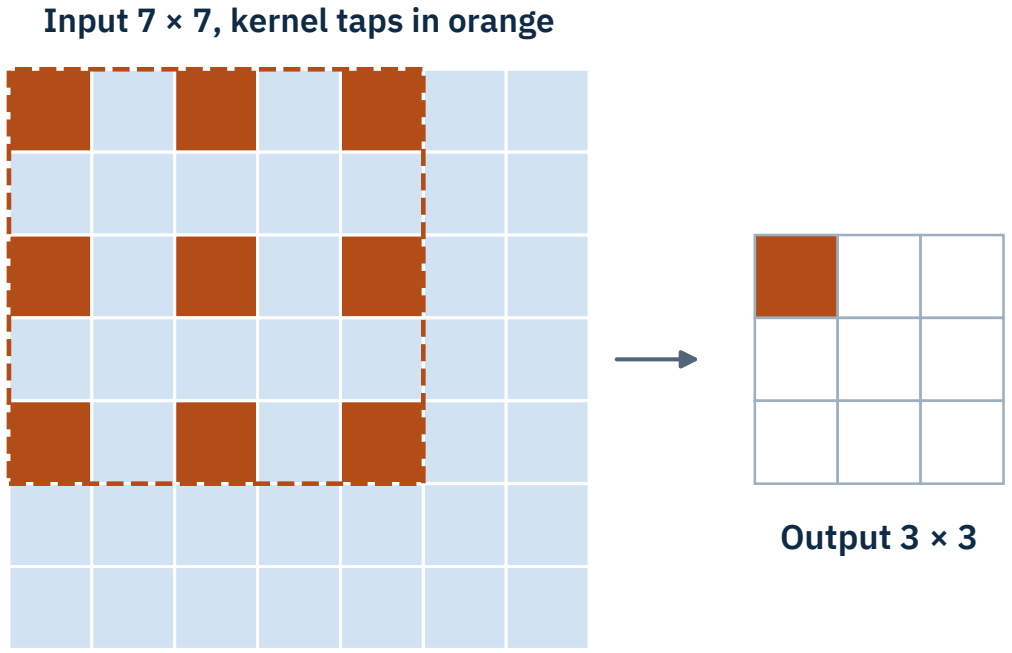


K = 3, S = 2, P = 0: $\lfloor (5 - 3)/2 \rfloor + 1 = 2$

Schematic; three of the four window positions shown.

Dilation widens a kernel's reach, not its weights

- **Dilation** D spaces the kernel taps D pixels apart.
- Nine weights cover a 5 × 5 area when D = 2.
- It adds context without downsampling: segmentation, seismic, 1-D signals.



K = 3, D = 2: taps span D(K – 1) + 1 = 5 pixels

Still 9 weights; (7 – 5)/1 + 1 = 3 outputs per row

Schematic; dilation 2, no padding, stride 1.

Output size follows kernel, stride, padding and dilation

$$H_{out} = \left\lfloor \frac{H_{in} + 2P - D(K-1) - 1}{S} + 1 \right\rfloor$$

- K kernel size, S stride, P padding, D dilation; the same rule gives the width.
- H_{in} = 32: K = 3, P = 1 keeps 32; adding S = 2 gives 16; K = 5, P = 0 gives 28.
- The number of output channels is the number of filters.

PRACTICE

4 min · Individual

Predict the shape before you run the layer

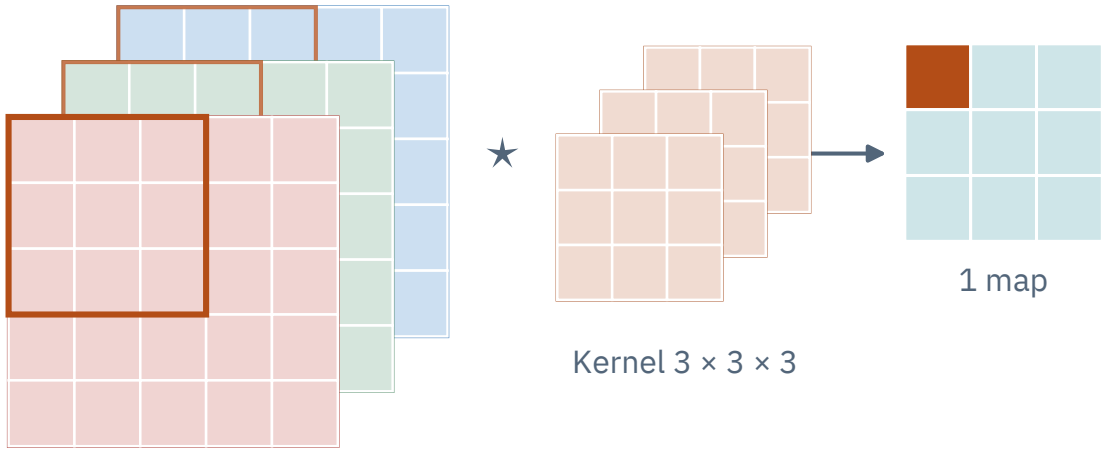
Input: B × 3 × 32 × 32 in PyTorch. Layer: `Conv2d(3, 16, kernel_size=3, stride=2, padding=1)`.

1. State the output shape.
2. Count the trainable parameters, including biases.
3. What changes, and what does not, if the input becomes 64 × 64?

Each kernel spans all input channels and makes one map

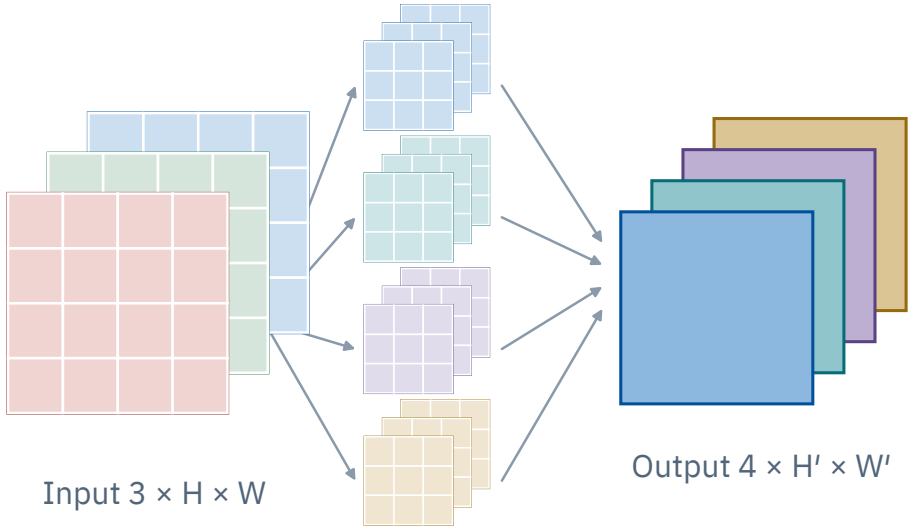
A Conv2d weight tensor has shape (C_{out}, C_{in}, K, K) and $C_{out}\,(C_{in}\,K^2 + 1)$ parameters.

One kernel spans every input channel



27 weights + 1 bias, summed over all channels

C_{out} kernels make C_{out} maps



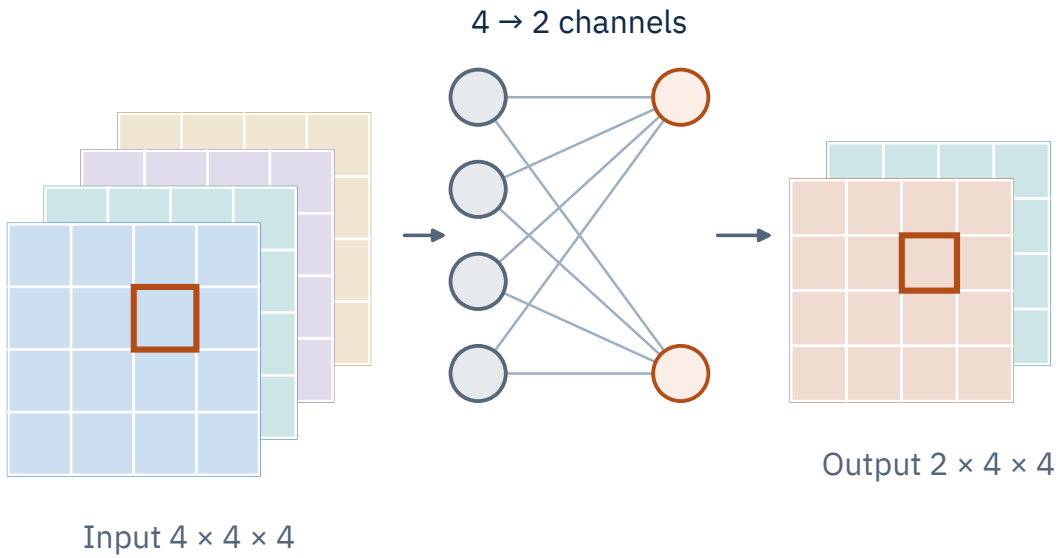
weight shape $(C_{out}, C_{in}, K, K) = (4, 3, 3, 3)$

Schematic; one bias per output channel.

A 1 × 1 convolution mixes channels at each pixel

- A **1 × 1 convolution** is a dense layer applied at every pixel.
- It changes the channel count cheaply: C_{out} (C_{in} + 1) parameters.
- Bottleneck blocks use it before costly 3 × 3 layers.

The same small dense layer at every pixel



Parameters: 2 × (4 + 1) = 10; H and W unchanged

Schematic: four channels mapped to two at every pixel.

PRACTICE

5 min · Pairs

Count the parameters before you build

Count weights and biases for each layer:

1. `Conv2d(64, 128, kernel_size=3, padding=1)`
2. `Conv2d(64, 16, kernel_size=1)`
3. Two stacked 3 × 3 layers (64 → 64 channels) versus one 5 × 5 layer (64 → 64). Which sees further?

DISCUSSION

Two 3 × 3 layers see as far as one 5 × 5 layer

Answers for 64 input and 64 output channels, biases included:

73,856

two stacked 3 × 3 layers

102,464

one 5 × 5 layer

5 × 5

receptive field of both

◆ The stack also puts a ReLU between its layers: more nonlinearity for fewer weights.

Convolution is a dense layer with two assumptions

Nearby values belong together, and useful patterns recur across positions.

Right for images and ordered signals; wrong for shuffled table columns.

PART 3 OF 7

03

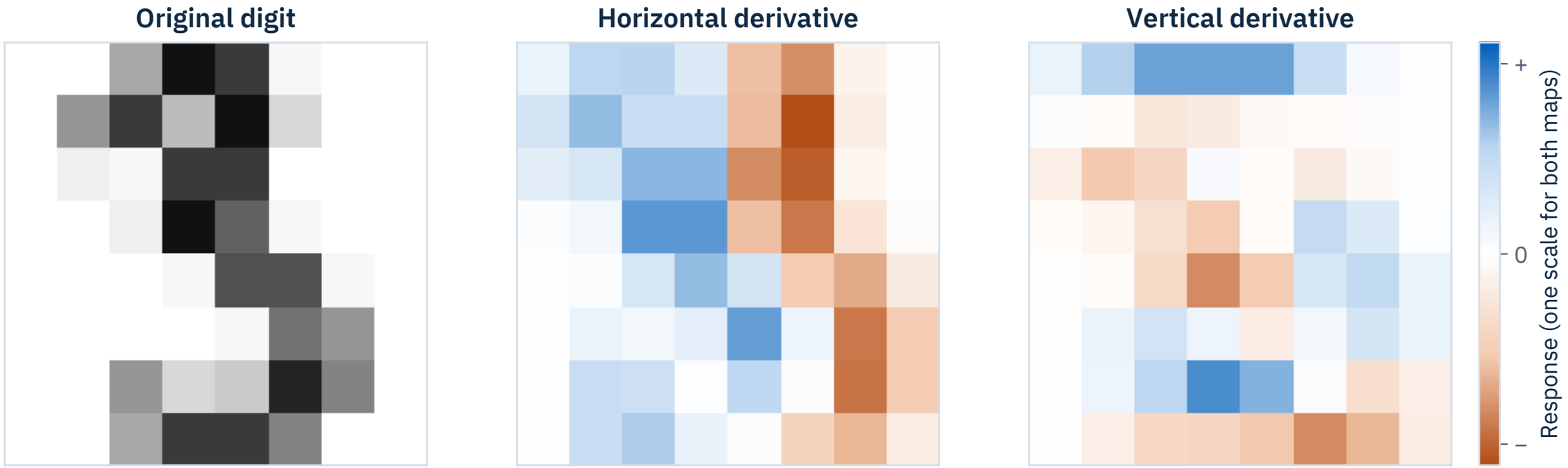
Feature maps and a compact CNN

Stacked filters see wider context; a ledger checks every shape.

- Hand-set and learned filters
- Receptive fields
- Pooling, strided convolution, global averaging
- Equivariance, invariance and the Lab 1 model

Filters respond to local structure

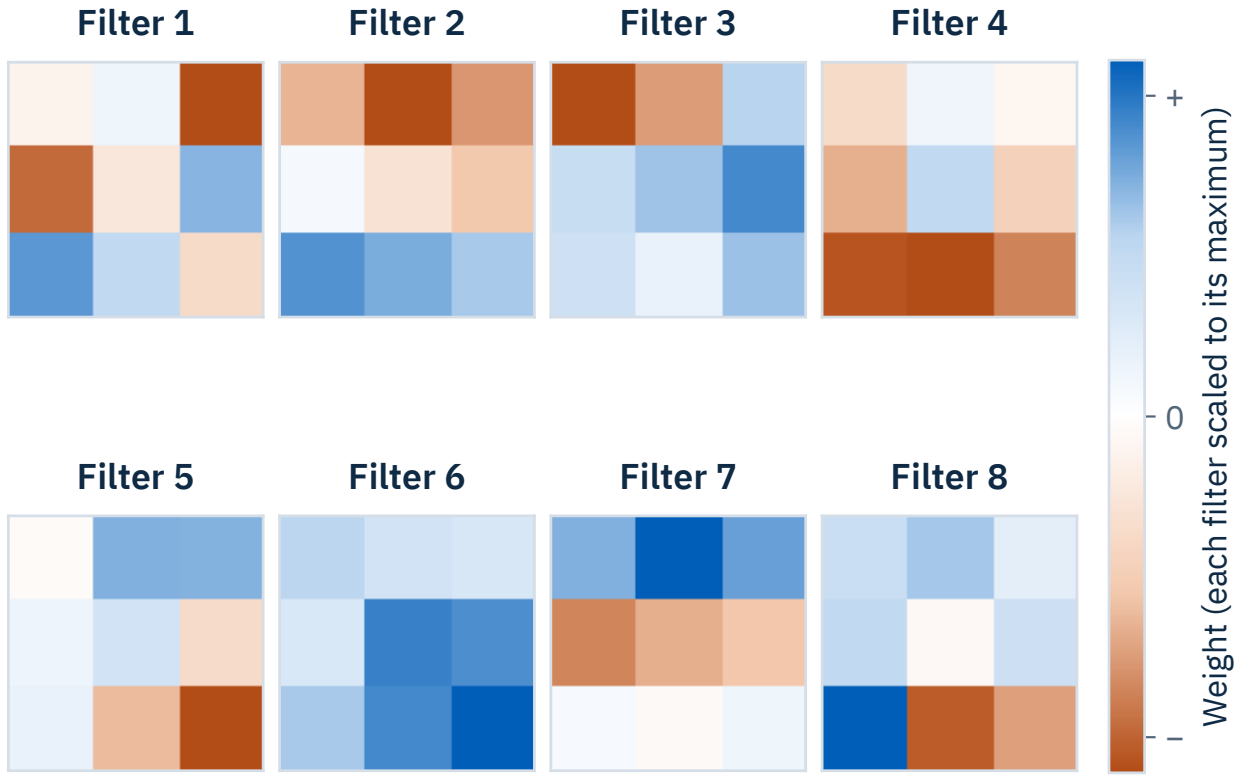
Sobel kernels respond where intensity changes; blue and orange mark opposite signs.



Fixed Sobel kernels applied to scikit-learn digit 3 with zero padding; hand-specified, not learned; one scale centred on zero.

Learned first-layer filters act as local detectors

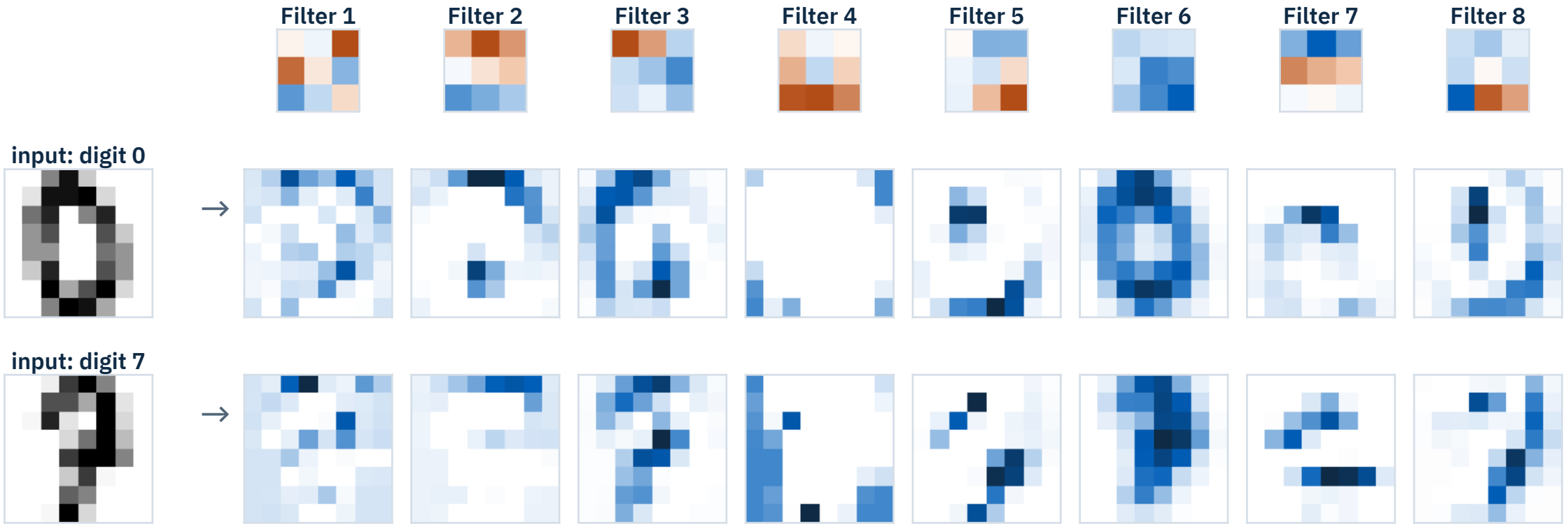
- A tiny CNN trained on 1,437 digits learned these eight 3 × 3 kernels.
- Several resemble oriented edge or stroke detectors.
- Not every filter has a human name; another seed gives different filters.



Tiny CNN (1,418 parameters) trained on scikit-learn digits, seed 0, CPU; 95.0% accuracy on 360 held-out digits.

Feature maps show where each filter fires

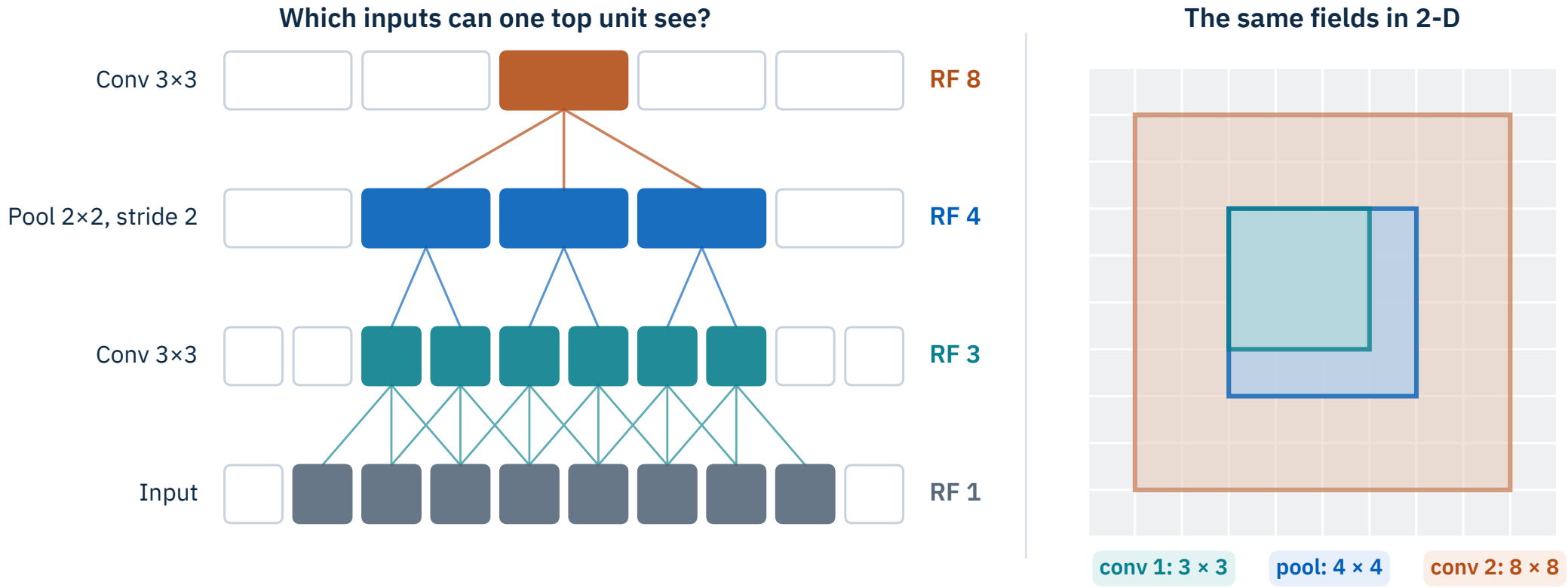
Maps keep the image layout: dark cells show where each filter's pattern appears.



First-layer ReLU maps of the tiny CNN for two held-out digits; each column scaled to its own maximum.

Receptive fields grow with depth

After conv 3 × 3, pool 2 × 2 and conv 3 × 3, one unit depends on an 8 × 8 patch of the input.



Exact receptive-field bookkeeping for stride-1 convolutions and stride-2 pooling.

A receptive-field ledger tracks jump and size

Layer	Kernel / stride	Jump j	Receptive field r
Input	–	1	1
Conv 1	3 / 1	1	3
Pool	2 / 2	2	4
Conv 2	3 / 1	2	8

Each layer applies $r \leftarrow r + (K - 1)\, j$, then $j \leftarrow j \cdot S$, with kernel K and stride S.

PRACTICE

4 min · Pairs

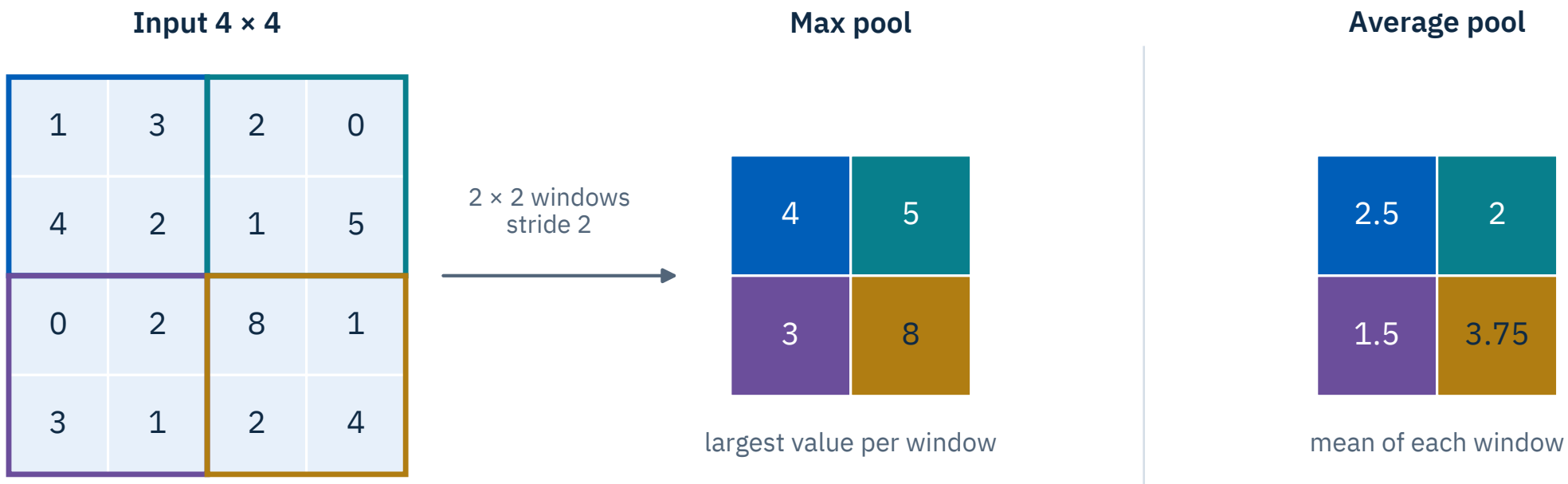
Extend the receptive-field ledger

Continue the table after Conv 2:

1. Add a pool 2 / 2, then a conv 3 / 1. Give j and r after each.
2. How many stride-1 3 × 3 convolutions alone would cover at least the same field?
3. Each depth row spans 0.1 m. How tall is that field in metres?

Pooling summarizes neighbourhoods

Max pooling keeps the strongest response in each window; average pooling keeps the mean.



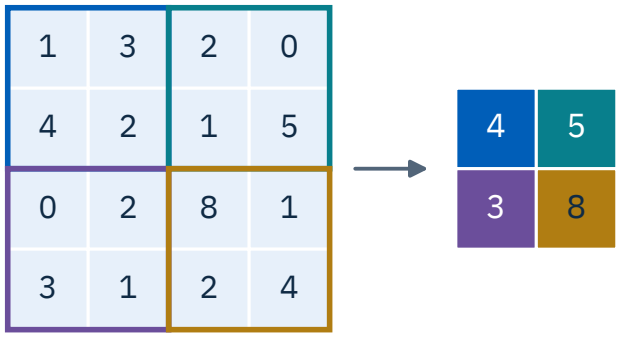
Exact max and average pooling on one 4 × 4 matrix, 2 × 2 windows, stride 2; colours link each window to its output.

Strided convolution and global averaging also downsample

Modern CNNs often use strided convolutions and end with global average pooling.

Max pool 2 × 2, stride 2

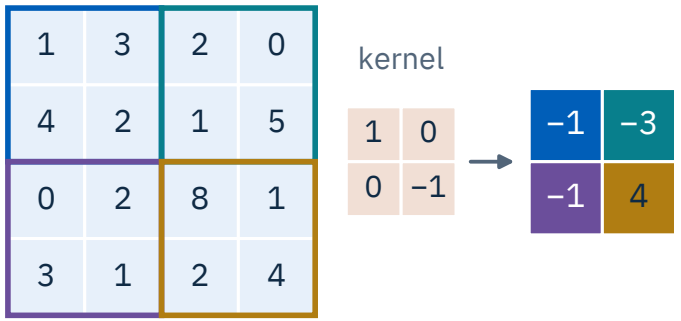
fixed rule, 0 parameters



keeps the largest value per window

Strided convolution 2 × 2, stride 2

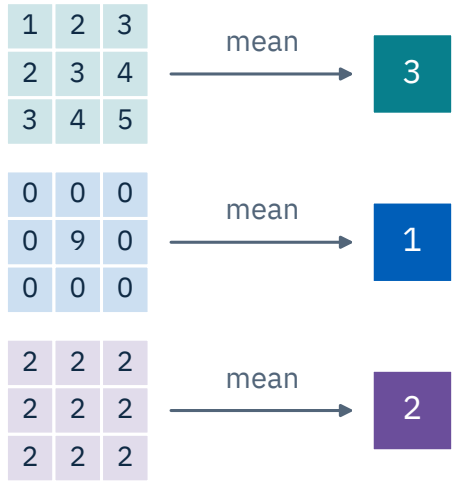
learned: 4 weights + 1 bias



weighted sum per window, learned

Global average pooling

one number per channel

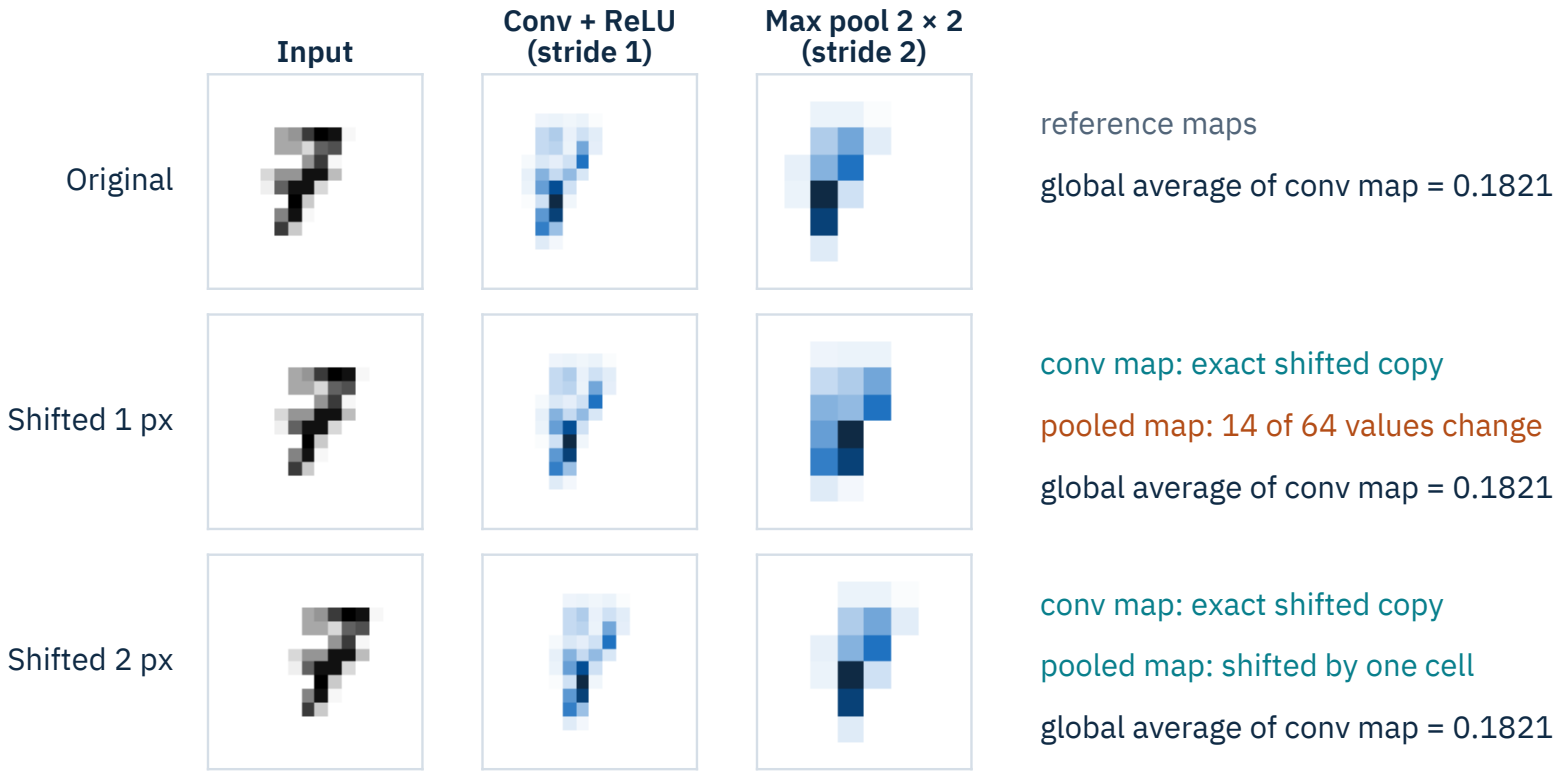


C × H × W → C, for any H × W

Exact arithmetic; the strided convolution uses kernel [[1, 0], [0, −1]] and zero bias.

Equivariance and invariance are different

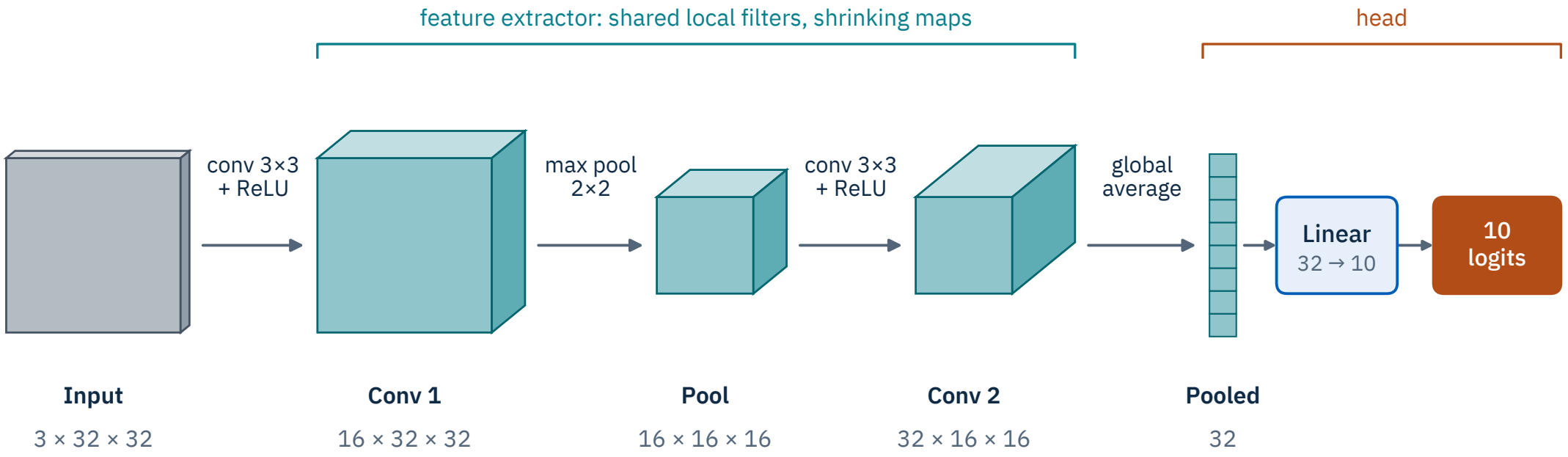
Shifted input, shifted map: **equivariance**. Same average: **invariance**.



Exact computation: a digit on a 16 × 16 zero canvas, fixed Sobel kernel with zero padding, ReLU, 2 × 2 max pooling.

A CNN alternates feature extraction and downsampling

Spatial size shrinks while channels grow; a small head turns pooled features into class logits.



Schematic of the compact CNN on the next slides; shapes are C × H × W for one image.

A shape ledger records every tensor in the model

Operation	Output for one batch	Purpose
Input	B × 3 × 32 × 32	RGB images
Conv 3 × 3, 16 filters, padding 1	B × 16 × 32 × 32	Local features
ReLU, max pool 2 × 2	B × 16 × 16 × 16	Nonlinearity, downsampling
Conv 3 × 3, 32 filters, padding 1, ReLU	B × 32 × 16 × 16	Feature combinations
Global average pool, linear	B × 10	Class logits

The compact CNN fits in one Sequential block

```
from torch import nn

model = nn.Sequential(
    nn.Conv2d(3, 16, 3, padding=1), nn.ReLU(),
    nn.MaxPool2d(2),
    nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(),
    nn.AdaptiveAvgPool2d(1),
    nn.Flatten(), nn.Linear(32, 10),
)
criterion = nn.CrossEntropyLoss()
```

No softmax before `CrossEntropyLoss`: it takes raw logits and integer class labels 0 to 9.

Run a dummy batch before any training

```python
import torch

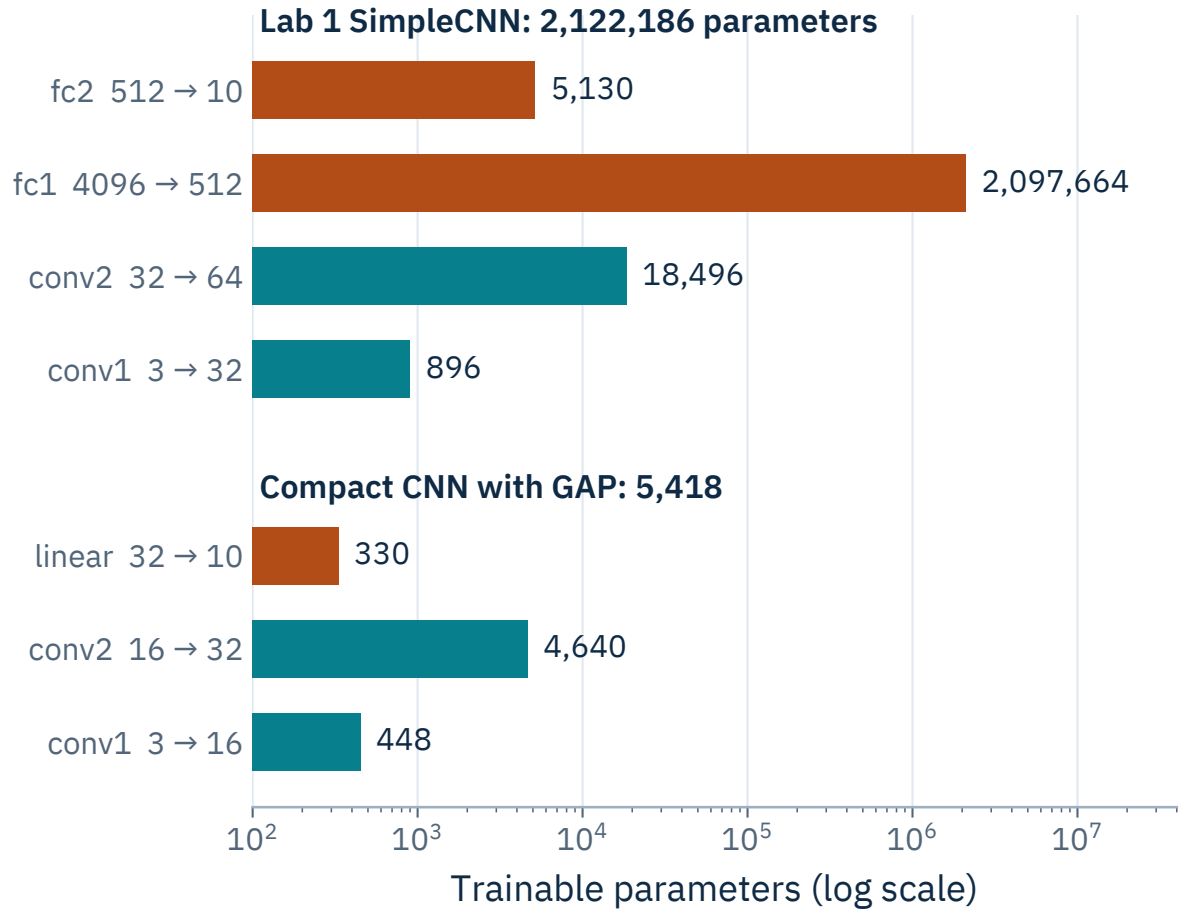
x = torch.randn(8, 3, 32, 32)
for layer in model:
    x = layer(x)
    print(f"{layer.__class__.__name__:<18} {tuple(x.shape)}")
print(sum(p.numel() for p in model.parameters()))
```

OUTPUT

```
Conv2d             (8, 16, 32, 32)
ReLU               (8, 16, 32, 32)
MaxPool2d          (8, 16, 16, 16)
Conv2d             (8, 32, 16, 16)
ReLU               (8, 32, 16, 16)
AdaptiveAvgPool2d  (8, 32, 1, 1)
Flatten            (8, 32)
Linear             (8, 10)
5418
```

Parameters concentrate in dense heads

- Lab 1's SimpleCNN has 2,122,186 parameters; 98.8% sit in one dense layer.
- Global average pooling shrinks the head to 330 parameters.
- Fewer parameters is not automatically better: compare on validation data.



Exact counts (weights and biases), checked with PyTorch.

LABORATORY

60 min · Pairs

Laboratory 1: a checked CNN on CIFAR-10

- Print every layer's output shape for one batch; count the parameters.
- Hold out 5,000 training images for validation; keep the test set for the end.
- Train SimpleCNN and plot training and validation loss and accuracy.
- Change one thing (BatchNorm and dropout, or a GAP head) and compare on validation.
- Inspect per-class accuracy and eight misclassified images.

NOTEBOOK

`exercises/01_cnn_cifar10/CNN_CIFAR10_Student.ipynb`

EVIDENCE TO SUBMIT

- Shape printout and parameter count
- Training and validation curves
- One controlled comparison, three failure cases

Prepare: torchvision installed and CIFAR-10 (about 170 MB) in ./data. Alternative: exercises/02_cnn_svhn (separate download).

DISCUSSION

Lab 1 debrief: read the curves and the errors

Shapes

- 64 × 8 × 8 = 4,096 inputs must match fc1
- A GAP head removes that coupling

Curves

- Is the train–validation gap growing?
- Compare models at the same epoch budget

Errors

- Which classes are confused, and do they look alike?
- What does 32 × 32 resolution hide?

Evidence

- One change per comparison
- Test set used once, at the end

PART 4 OF 7

04

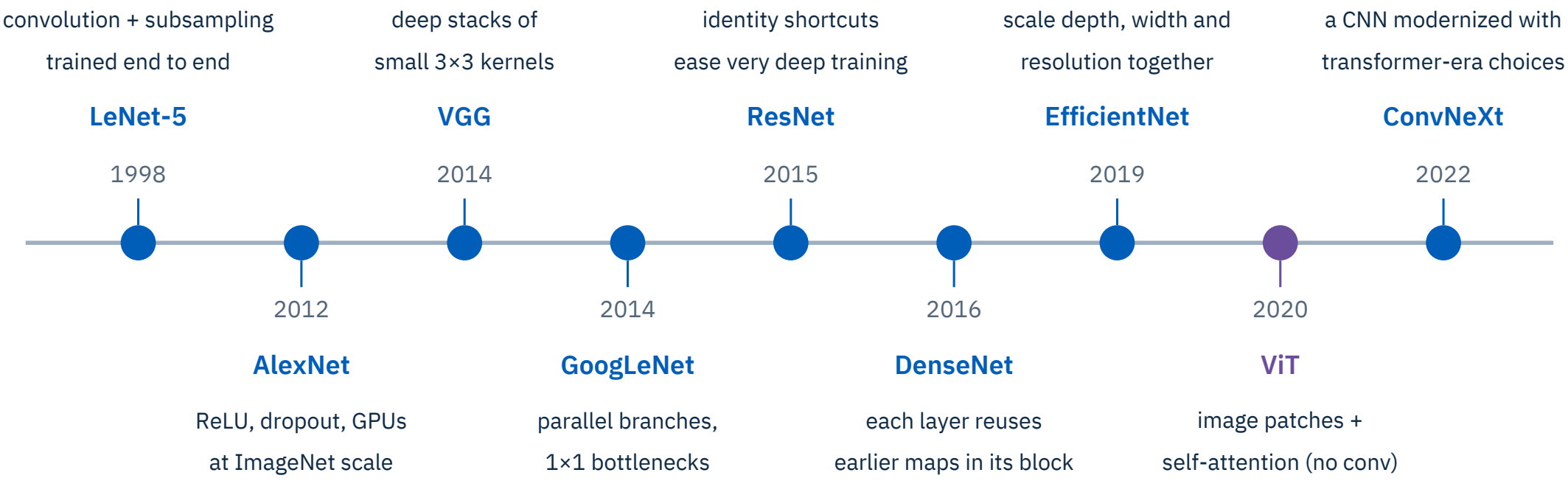
Architecture ideas

From LeNet to ConvNeXt, one reusable design idea at a time.

- A timeline of design ideas
- Residual connections
- Batch normalization in CNNs
- Choosing a backbone

Architectures evolved by adding reusable ideas

Read the history as a toolbox, not a leaderboard: each name contributed an idea still in use.



Schematic timeline, not to scale; years of first publication (conference or preprint); no benchmark results.

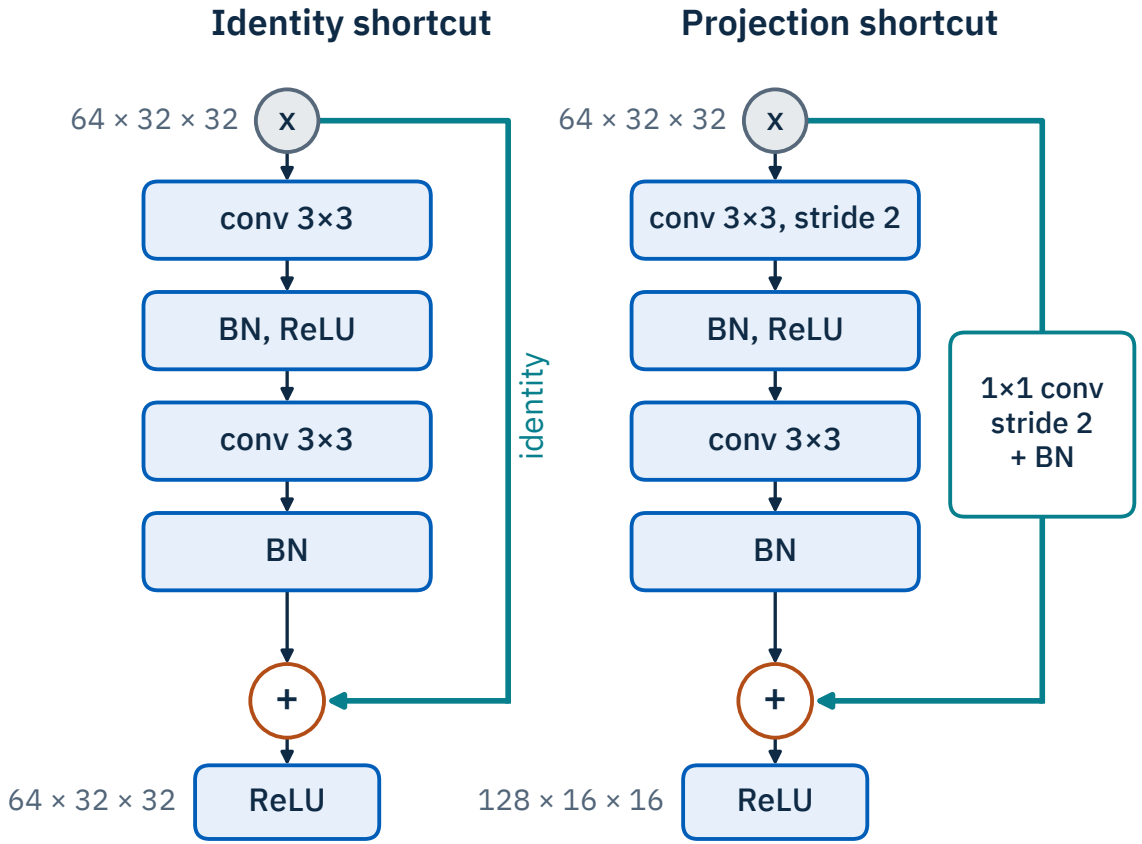
Each architecture family leaves a practical lesson

Family	Idea to retain	Practical lesson
LeNet / AlexNet	Learned convolutional feature hierarchies	Local features can support recognition
VGG	Repeated small kernels	Depth trades compute for capacity
Inception	Parallel branches, 1 × 1 bottlenecks	Mix scales at modest cost
ResNet / DenseNet	Shortcut paths between layers	Identity paths help optimization
MobileNet / EfficientNet	Efficiency-oriented design	Match capacity to deployment constraints

Residual connections preserve an identity path

$$y = x + F(x)$$

- The branch *F* learns a correction to its input.
- Addition needs matching shapes, or a 1 × 1 projection shortcut.
- The identity route helps gradients and signals pass through deep stacks.



Schematic: identity shortcut (left) and projection shortcut (right).

PRACTICE

4 min · Pairs

Make the residual addition shape-compatible

Input x is $64 \times 32 \times 32$. The branch starts with a 3×3 convolution, stride 2, padding 1, 128 filters.

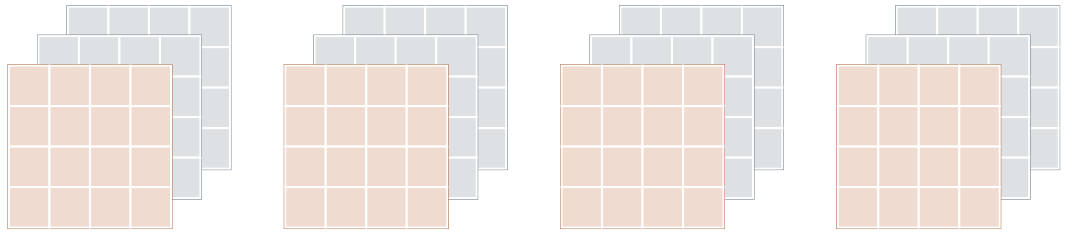
1. What shape does the branch output $F(x)$ have?
2. What must the shortcut do before the addition?
3. How many weights does a 1×1, stride-2 projection from 64 to 128 channels have, without bias?

Batch normalization standardizes each channel

- BatchNorm2d computes one mean and variance per channel over B × H × W.
- Learned scale γ and shift β restore flexibility.
- Evaluation uses running statistics, so train and eval modes differ.
- Tiny batches give noisy statistics; GroupNorm is one alternative.

One mean and variance per channel

each image: C = 3 channel sheets, channel c in front



channel c across the batch and positions: B × H × W values

$$\hat{x} = (x - \mu_c) / \sqrt{\sigma_c^2 + \varepsilon}, \quad y = \gamma_c \hat{x} + \beta_c$$

BatchNorm2d(C): 2C learned (γ, β) + 2C running statistics

Schematic of BatchNorm2d statistics; normalization in general is covered on Day 2.

Frozen weights do not freeze BatchNorm statistics

```python
import torch
from torch import nn

torch.manual_seed(0)
bn = nn.BatchNorm2d(3)
for p in bn.parameters():
    p.requires_grad = False        # gamma and beta frozen

x = torch.randn(16, 3, 8, 8) * 2 + 5
bn.train(); bn(x)                  # training mode: statistics update
print(bn.running_mean)
bn.eval()                          # evaluation mode: running statistics
print(f"{bn(x).mean():.2f}")
```

OUTPUT

```
tensor([0.5045, 0.4930, 0.5014])
3.94
```

ARCHITECTURE IDEAS

Choose a backbone from data, compute and evidence

Little labelled data

- Start from pretrained features
- Freeze most of the backbone

Strong domain shift

- Fine-tune the later stages
- Validate on held-out wells or surveys

Edge or real-time use

- Efficient families such as MobileNet or EfficientNet
- Measure latency on the target hardware

Ample data and compute

- Larger models become affordable
- Still compare with a small baseline

◆ No architecture choice replaces a grouped validation design.

PRACTICE

3 min · Individual

Match each design idea to the problem it solves

Choose from: residual connection, global average pooling, 1 × 1 bottleneck, batch normalization.

1. A deep plain stack trains worse than a shallower one.
2. A flatten-and-dense head holds 99% of the parameters.
3. 256 channels make every 3 × 3 layer expensive.
4. Activation scales drift between layers during training.

PART 5 OF 7

05

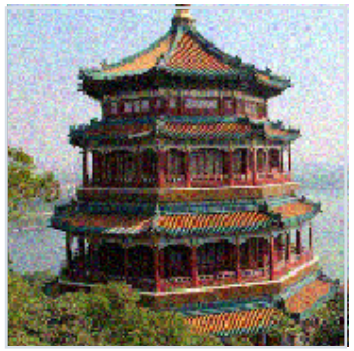
Training and transfer learning

Data, augmentation and honest splits first, then pretrained features.

- Augmentation that preserves the label
- Class imbalance in defect data
- Source-grouped evaluation
- Feature extraction and fine-tuning

Augmentation creates plausible new training views

Each transform encodes a claim: the label does not depend on this change.

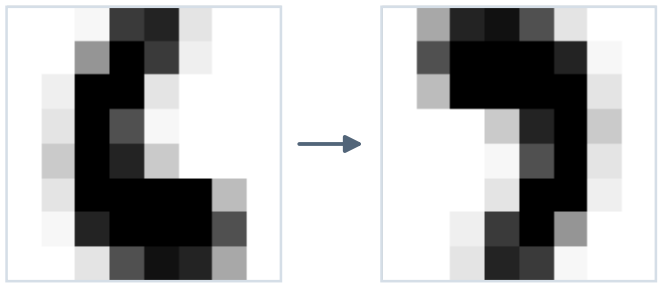
Original	Horizontal flip	Random crop	Rotation	Brightness	Gaussian noise
	p = 0.5	70% side, resized	15°, black corners	× 1.3, clipped	σ = 0.08
					
reference	*mirror symmetry*	*framing and scale*	*orientation*	*illumination*	*sensor noise*

scikit-learn sample image china.jpg (CC BY 2.0), 360 × 360 crop resized to 160 × 160; transforms computed with NumPy and SciPy, seed 3.

Some transforms silently change the label

Whether a transform is safe depends on the physics and on how the target is defined.

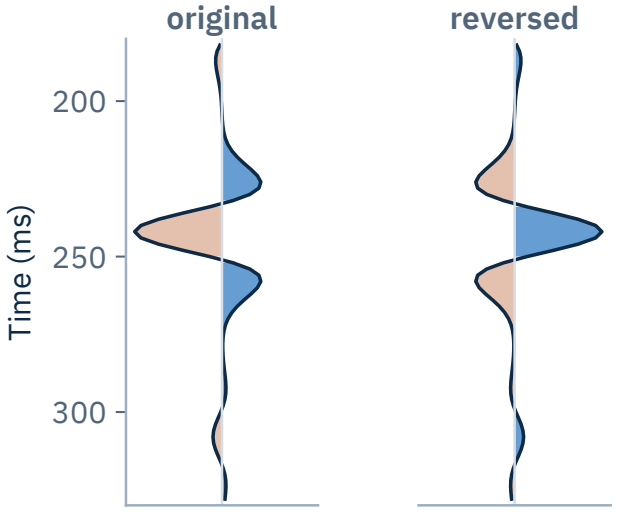
Rotate a “6” by 180°



label 6 → reads as 9

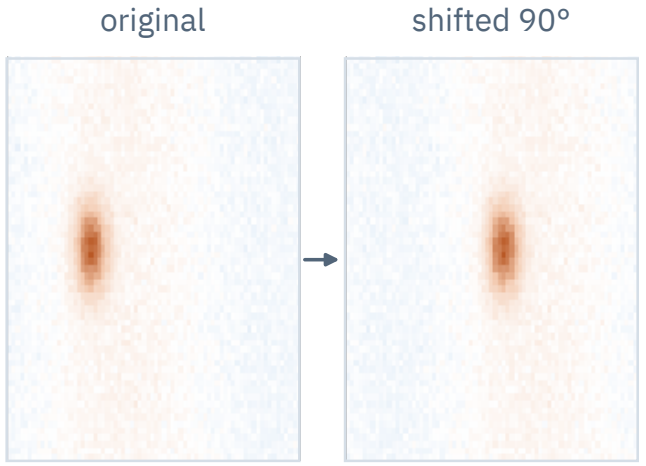
✗ label changes

Reverse seismic polarity



▲ label may change

Rotate a casing map (azimuth)



pits are still pits

✓ label preserved

scikit-learn digit; synthetic seismic trace (seed 7); synthetic thickness map (seed 5) shifted by 18 of 72 sectors.

PRACTICE

4 min · Pairs

Augmentation must preserve the task label

You classify casing defects from inspection images.

1. Is a horizontal flip physically meaningful for this acquisition geometry?
2. Could cropping remove the defect and leave its original label?
3. Which transforms belong only in training?

Training and evaluation use different transforms

```python
import torch
from torchvision.transforms import v2

MEAN, STD = [0.485, 0.456, 0.406], [0.229, 0.224, 0.225]
train_tf = v2.Compose([
    v2.ToImage(),
    v2.RandomResizedCrop(224, scale=(0.8, 1.0), antialias=True),
    v2.RandomHorizontalFlip(p=0.5),
    v2.ColorJitter(brightness=0.2),
    v2.ToDtype(torch.float32, scale=True),
    v2.Normalize(MEAN, STD),
])
eval_tf = v2.Compose([
    v2.ToImage(), v2.Resize(256, antialias=True), v2.CenterCrop(224),
    v2.ToDtype(torch.float32, scale=True), v2.Normalize(MEAN, STD),
])
```

ImageNet statistics shown; use those of your pretrained weights or of your training data.

Accuracy hides rare defects

5,000 inspection patches, 100 of them defective (2%). A model that always answers "normal":

98%	0%	0
accuracy	defect recall	defects found

- Report recall and precision per class and per well; Machine Learning Module 5 covers rare-event metrics.

Class weights rebalance the loss, not the data

```python
import torch
from torch import nn

y_train = torch.tensor([0] * 4900 + [1] * 100)
counts = torch.bincount(y_train, minlength=2)
weights = counts.sum() / (2 * counts.float())
print(weights)
criterion = nn.CrossEntropyLoss(weight=weights)
```

OUTPUT

```
tensor([ 0.5102, 25.0000])
```

- Each defective patch now weighs 49 times a normal one in the loss.
- Alternatives: WeightedRandomSampler, focal loss, threshold tuning.
- Choose the decision threshold on validation data.

A valid image evaluation separates sources

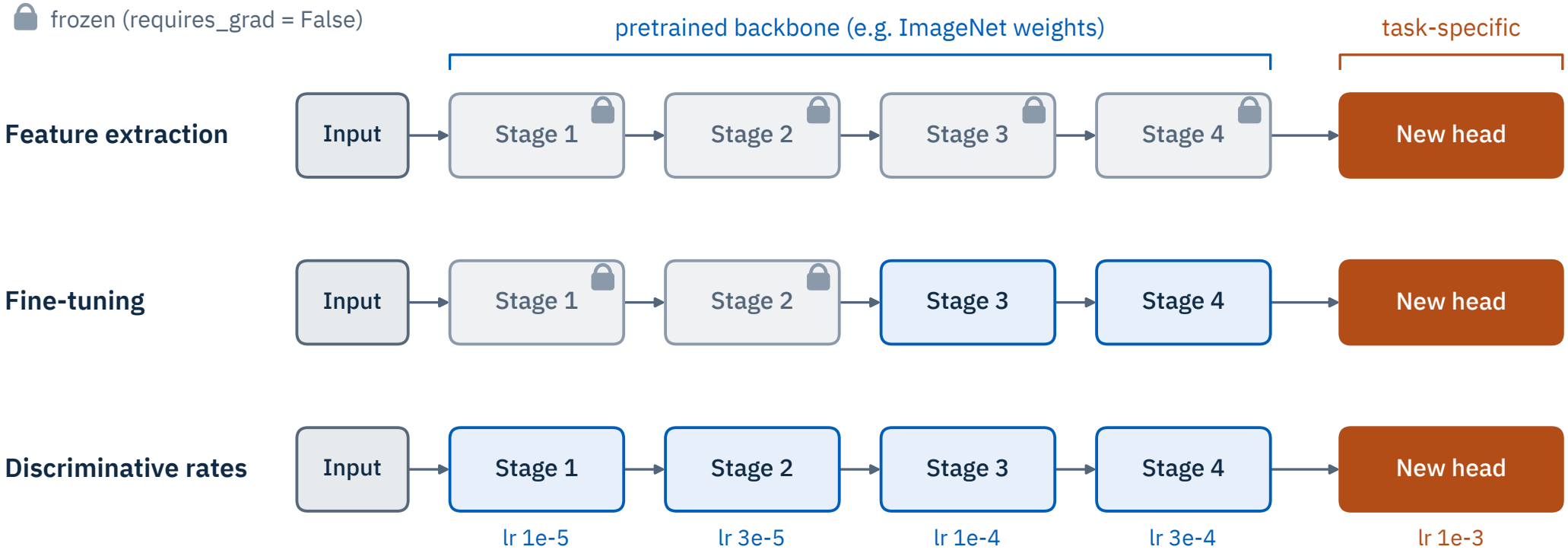
Overlapping windows are near-duplicates: split by well, survey or asset first.



Schematic: 32-row windows with stride 16 (50% overlap); random assignment drawn with seed 8.

Transfer learning reuses a pretrained backbone

Pretrained features are a hypothesis about the new domain; test it.



Schematic of three adaptation strategies; learning rates are illustrative.

Adaptation proceeds from frozen features to fine-tuning

Stage	Trainable parameters	Main question
Feature extraction	New task head	Are the existing features useful?
Fine-tuning	Head and later backbone stages	Does adaptation improve validation performance?
Discriminative rates	All, smaller rates near the input	Does gentler updating of early layers help?

Use the preprocessing that belongs to the chosen pretrained weights.

Load pretrained weights with their own preprocessing

```
from torchvision.models import resnet18, ResNet18_Weights
from torch import nn

weights = ResNet18_Weights.DEFAULT
model = resnet18(weights=weights)
for parameter in model.parameters():
    parameter.requires_grad = False
model.fc = nn.Linear(model.fc.in_features, n_classes)
preprocess = weights.transforms()
```

The new `fc` layer is created after freezing, so only its parameters train.

Discriminative learning rates use parameter groups

```python
import torch

for name, p in model.named_parameters():
    p.requires_grad = name.startswith(("layer4.", "fc."))

optimizer = torch.optim.AdamW([
    {"params": model.layer4.parameters(), "lr": 1e-4},
    {"params": model.fc.parameters(), "lr": 1e-3},
], weight_decay=1e-4)
```

Frozen stages stay out of the optimizer; keep their BatchNorm layers in eval mode while fine-tuning.

PRACTICE

5 min · Pairs

Choose an adaptation strategy for three datasets

For each dataset: feature extraction, fine-tuning or training from scratch? And how would you split it?

1. 2,000 labelled core photographs of six lithologies from 12 wells.
2. 300 labelled seismic patches from one survey, to be used on a second survey.
3. 50,000 labelled ultrasonic casing patches from 40 wells.

06

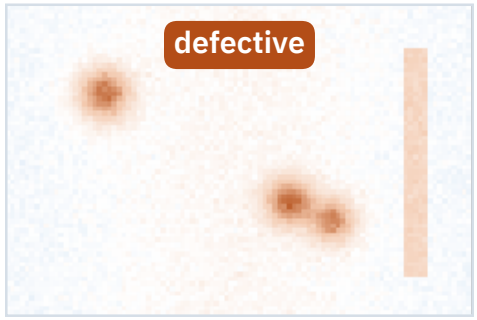
Beyond classification

The same features can return boxes, masks and evidence maps.

- Classification, detection, semantic and instance segmentation
- Intersection over union and U-Net
- Occlusion and gradient maps
- Shortcut learning

Vision tasks differ in what they return

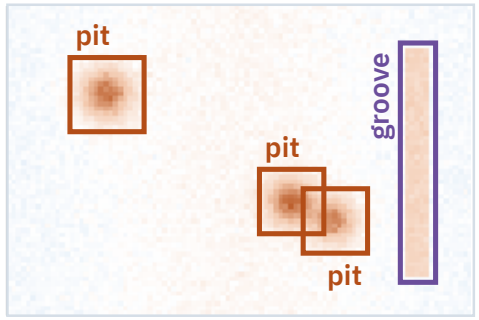
The label format decides the model, the loss and the metric.



Classification

one label per image

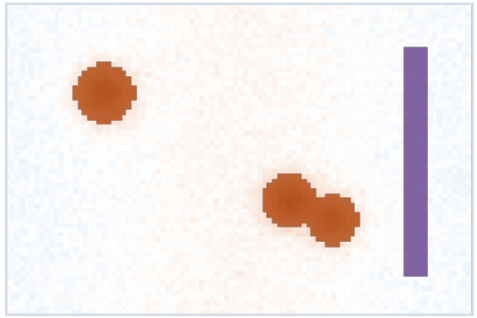
“defects present”



Object detection

box + class + score per object

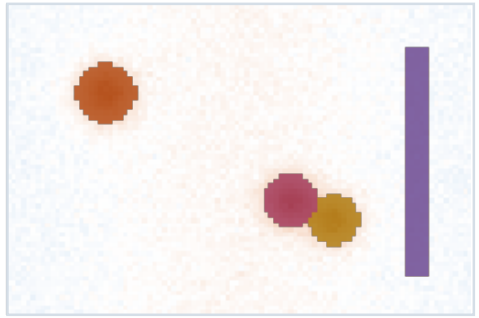
matched to truth by IoU



Semantic segmentation

a class for every pixel

pits merge into one region



Instance segmentation

a separate mask per object

touching pits stay separate

Synthetic thickness patch (seed 12) with three pits and a groove; labels derived from the known geometry, not from a model.

Object detectors predict boxes, classes and scores

A detector turns image features into a list of scored boxes.

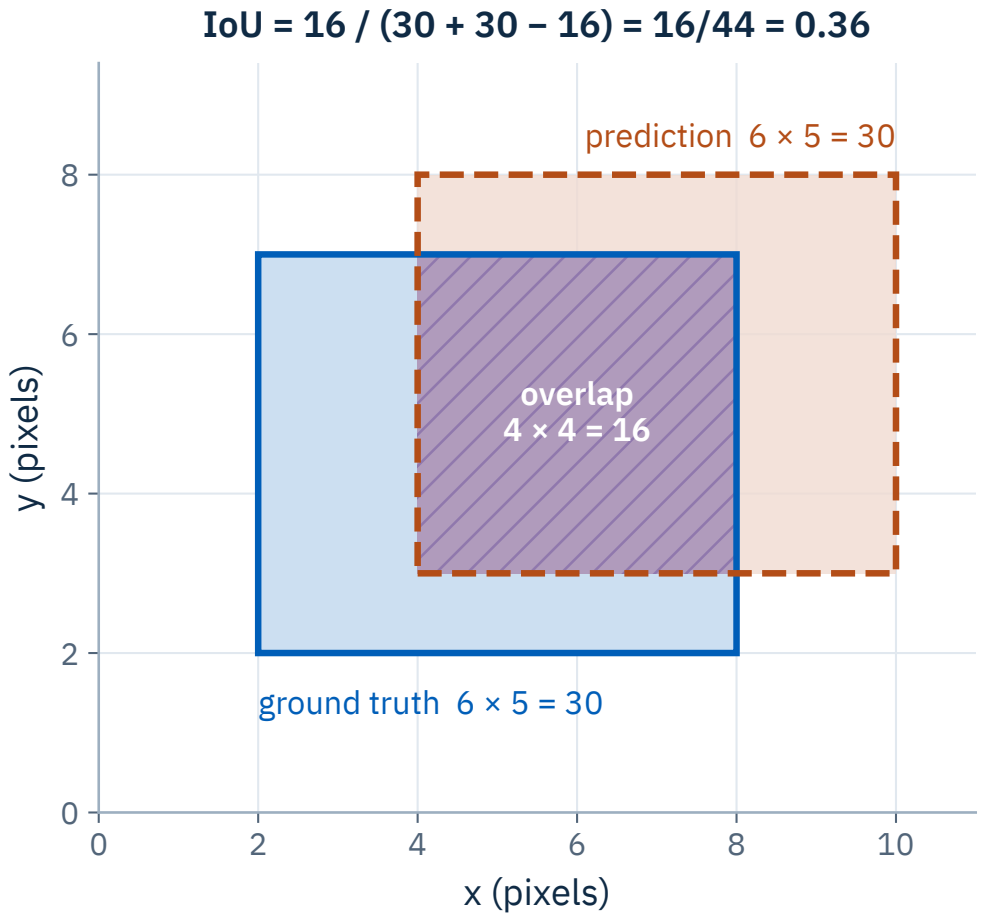
1. **Backbone**
   CNN features for the whole image
2. **Candidates**
   anchors, points or learned queries
3. **Heads**
   class scores and box offsets
4. **Clean-up**
   non-maximum suppression of duplicates
5. **Evaluation**
   match to ground truth by IoU

◆ Two-stage (Faster R-CNN), one-stage (YOLO, RetinaNet) and transformer (DETR) detectors share this output format.

Intersection over union scores box overlap

$$\mathrm{IoU} = \frac{|A \cap B|}{|A \cup B|}$$

- Here the overlap is 16 and the union 44: IoU = 0.36.
- A detection counts only above a threshold, often 0.5.
- COCO-style scores average over thresholds 0.50 to 0.95.



Exact intersection over union for two axis-aligned boxes.

PRACTICE

4 min · Pairs

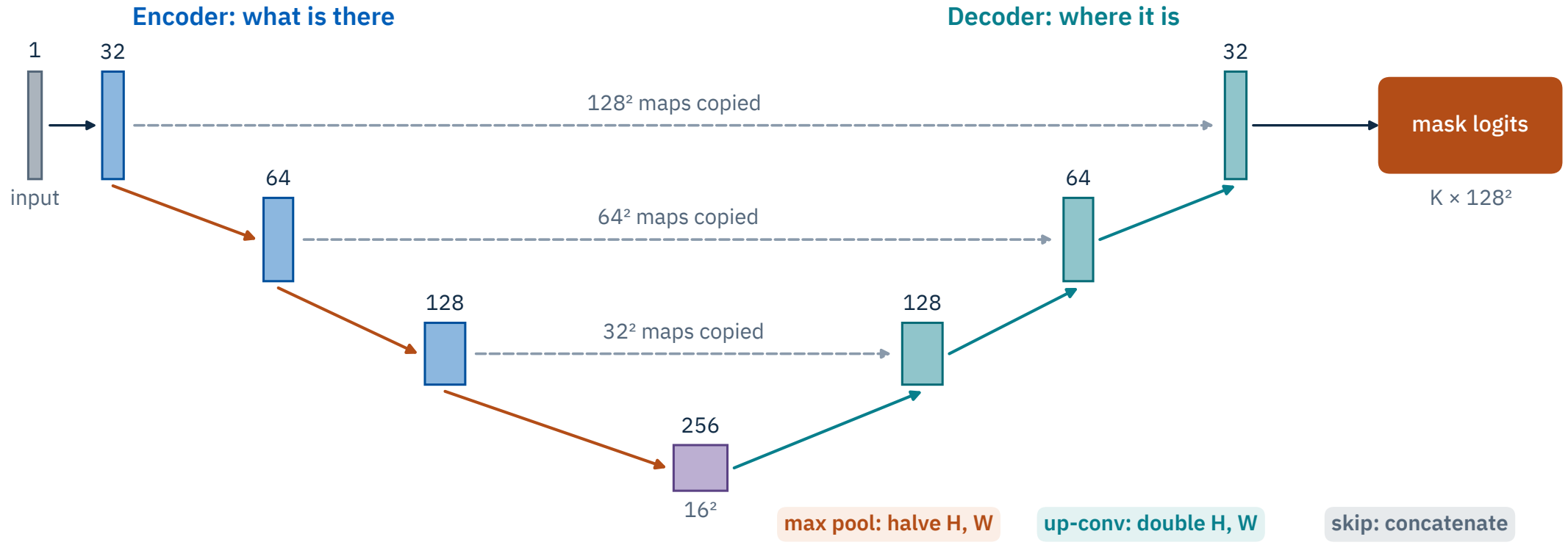
Decide whether each detection is a true positive

Ground-truth box corners: (0, 0) and (4, 4). A detection counts if IoU ≥ 0.5.

1. Detection A has corners (1, 1) and (5, 5). Compute its IoU.
2. Detection B has corners (0, 1) and (4, 5). Compute its IoU.
3. Both are predicted for the same defect. How many true and false positives?

U-Net segments with an encoder, decoder and skips

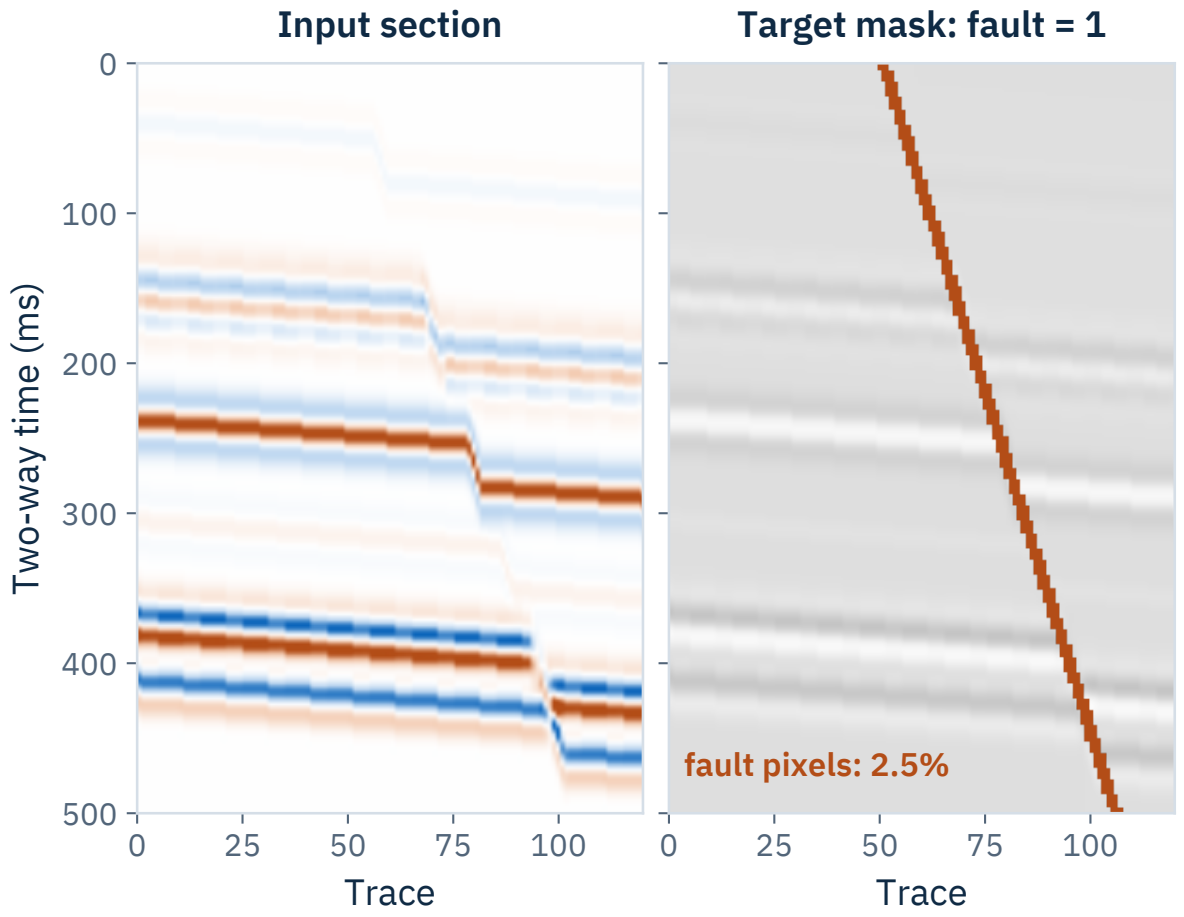
The encoder gathers context, the decoder restores resolution, and skips carry back fine detail.



Schematic of a small U-Net for a 128 × 128 input; numbers are channel counts. After Ronneberger et al. (2015).

Segmentation masks bring their own label problems

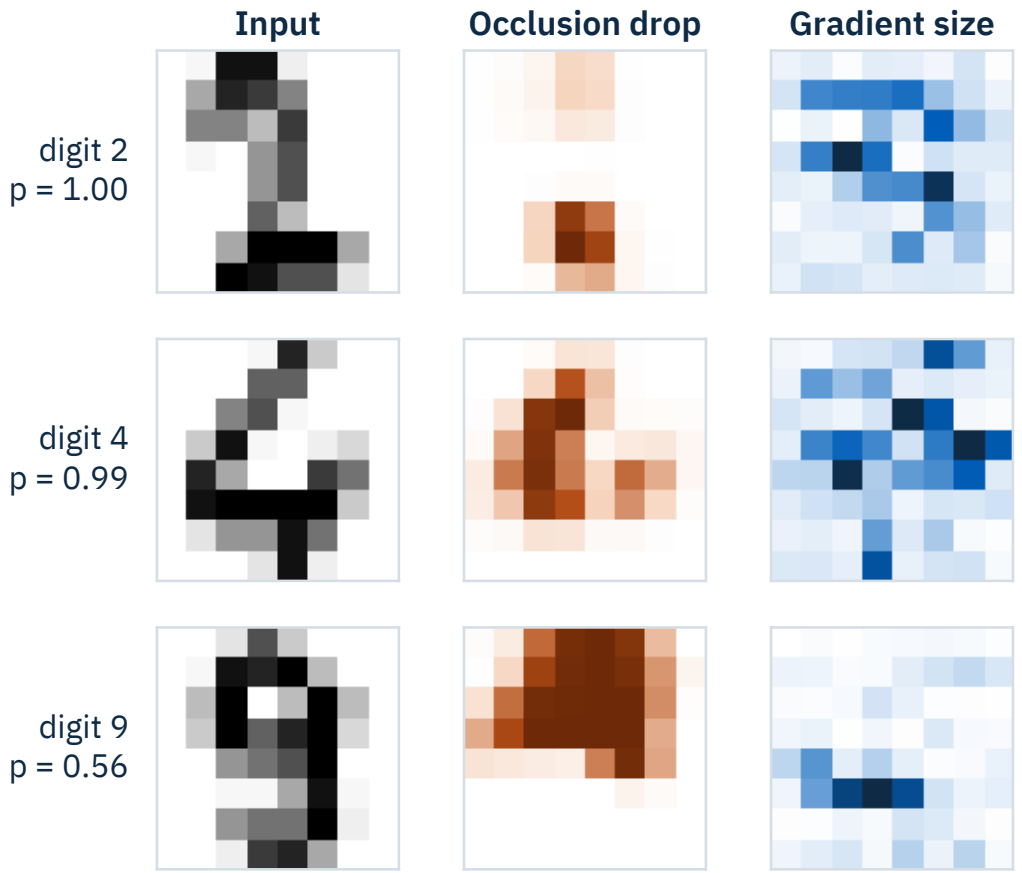
- Pixel labels are slow to draw and differ between interpreters.
- Targets are thin and rare: here 2.5% of pixels are fault.
- Score with Dice or IoU per class, with tolerance near boundaries.
- **Instance segmentation** (e.g. Mask R-CNN) also separates objects.



Synthetic section (seed 7); fault mask: pixels within 1.5 traces of the fault plane.

Occlusion maps show which pixels move the score

- **Occlusion**: blank a patch, record the drop in the class probability.
- Gradient maps: how strongly each pixel changes the class logit.
- Both measure local sensitivity; neither explains the reasoning.



Tiny digits CNN (seed 0): 2 × 2 patches set to 0; absolute input gradients; each map scaled to its maximum.

Shortcuts can pass validation and fail in deployment

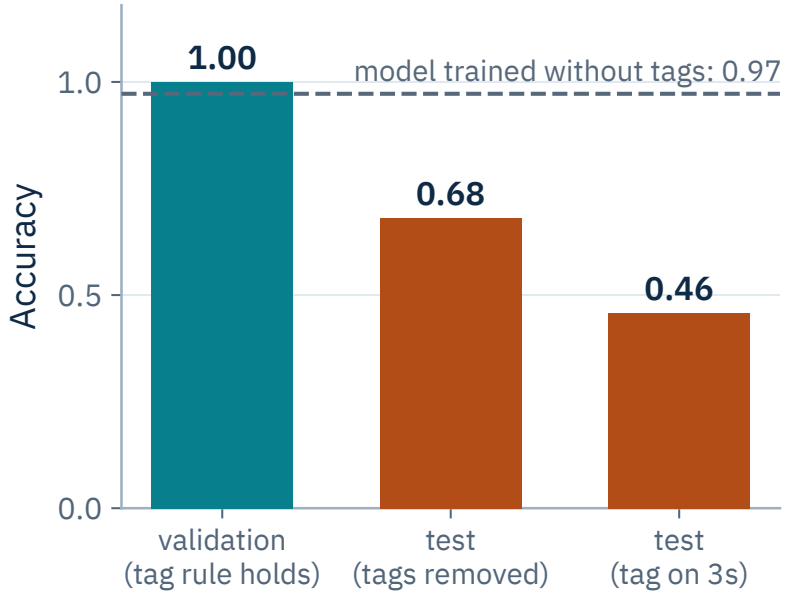
A bright tag on every training 8 is enough: validation is perfect, then accuracy collapses.

Training data

every 8 carries a corner tag

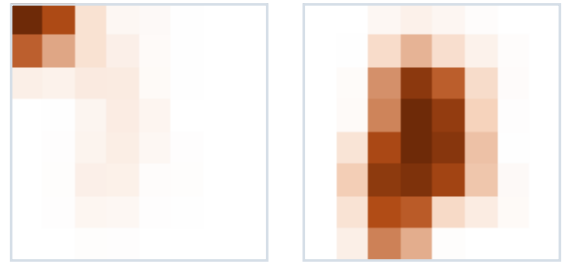


Accuracy of the tagged model



Where the evidence is

occlusion drop for a tagged 8



tagged model

p(8) = 1.00

clean model

p(8) = 0.99

Tiny CNN (seed 0, CPU) on scikit-learn 3s and 8s: 214 training, 71 validation and 72 test images.

Explanations are hypotheses to test

✕ Shortcut sources in energy images

- Tool, run or vendor artefacts that correlate with labels
- Depth tracks, annotations or scale bars in the image
- One well or survey supplying most positives

vs

✓ Tests that expose them

- Grouped splits by well, survey or acquisition run
- Counterfactual edits: remove the tag, move the defect
- Expert review of the evidence maps

PART 7 OF 7

07

Autoencoders and 1-D signals

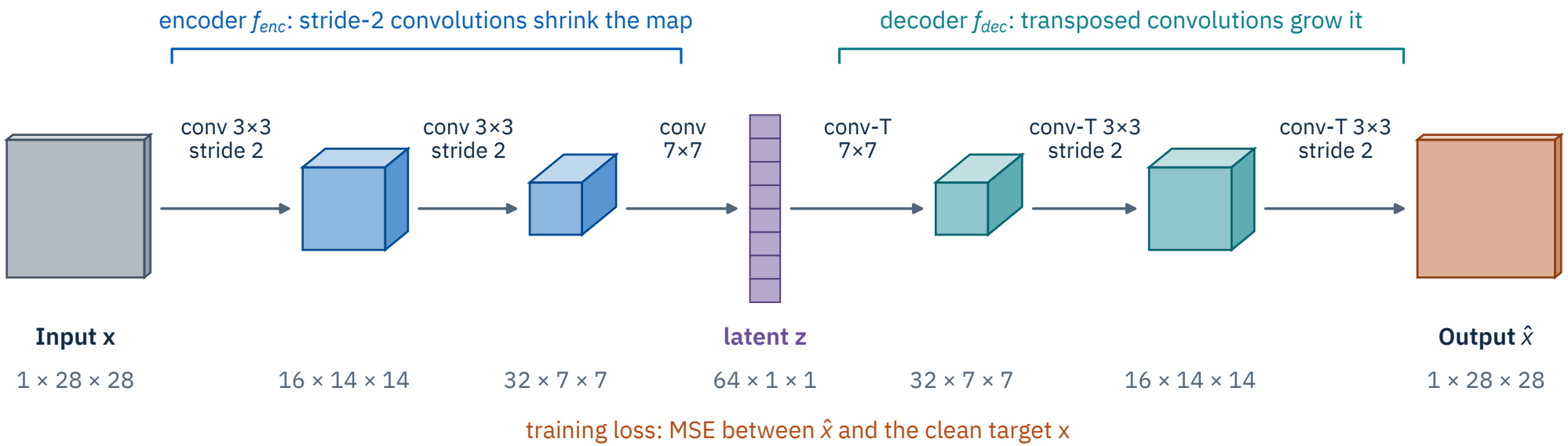
The output can be a reconstruction or an ordered signal.

- Reconstruction objective and denoising
- Reconstruction error for anomaly screening
- 1-D convolutions for sensor series
- Energy application choices

An autoencoder learns a reconstruction objective

$$z = f_{enc}(x), \qquad \hat{x} = f_{dec}(z), \qquad L = \|x - \hat{x}\|^2$$

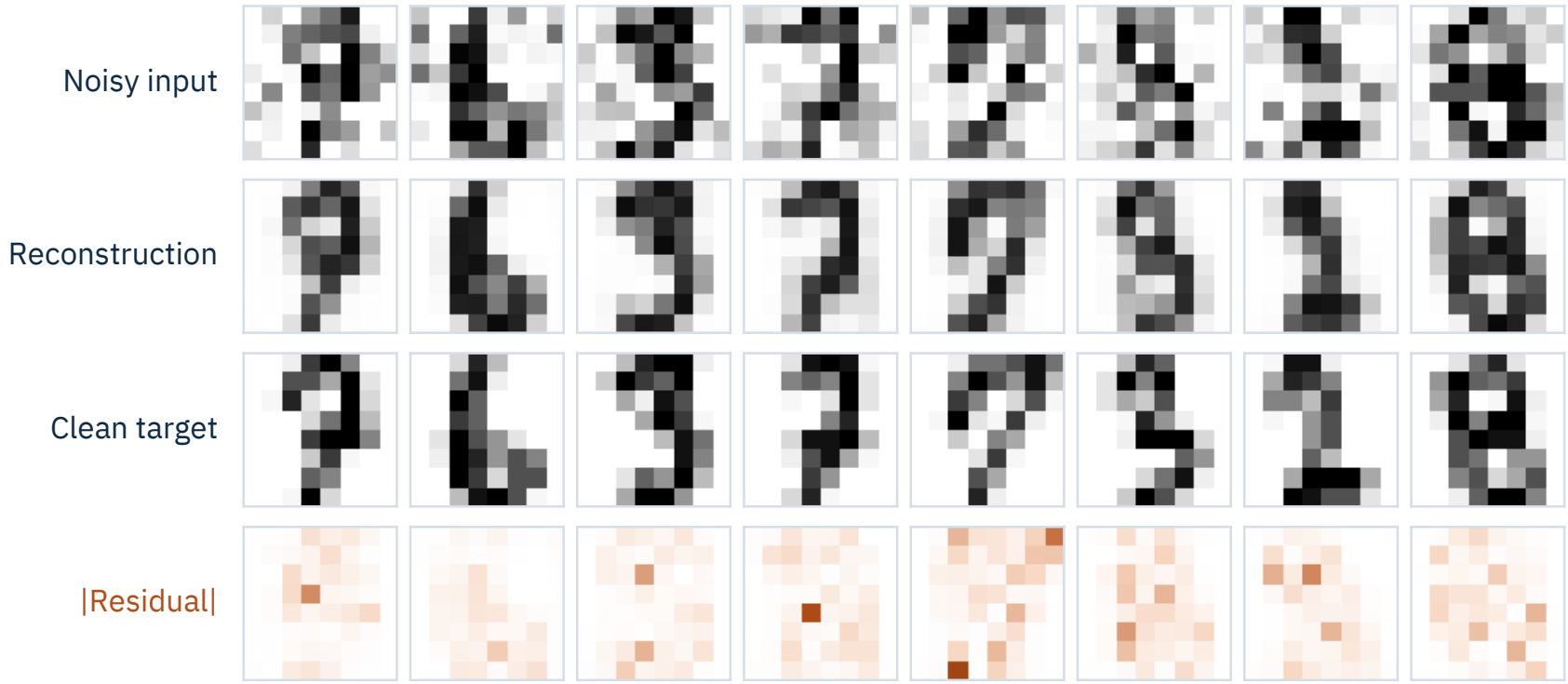
A bottleneck prevents copying; denoising maps corrupted inputs to clean targets.



Schematic of the Lab 2A autoencoder (latent size 64); shapes checked with PyTorch, batch dimension omitted.

Denoising trains from corrupted input to clean target

Held-out error against the clean digits falls from 0.051 for the noisy input to 0.019.



Held-out MSE

vs clean target

noisy input 0.051

reconstruction 0.019

9,569 parameters

noise σ = 0.3, clipped

Small convolutional denoising autoencoder (9,569 parameters, seed 0, CPU) trained on scikit-learn digits; noise σ = 0.3, clipped.

Reconstruction quality needs more than a low loss

Inspect residuals

- Compare input, target, reconstruction, residual
- Structure in residuals means missed signal

Protect small features

- Check whether task-relevant defects vanish
- A smoother image can be less useful

Keep scaling consistent

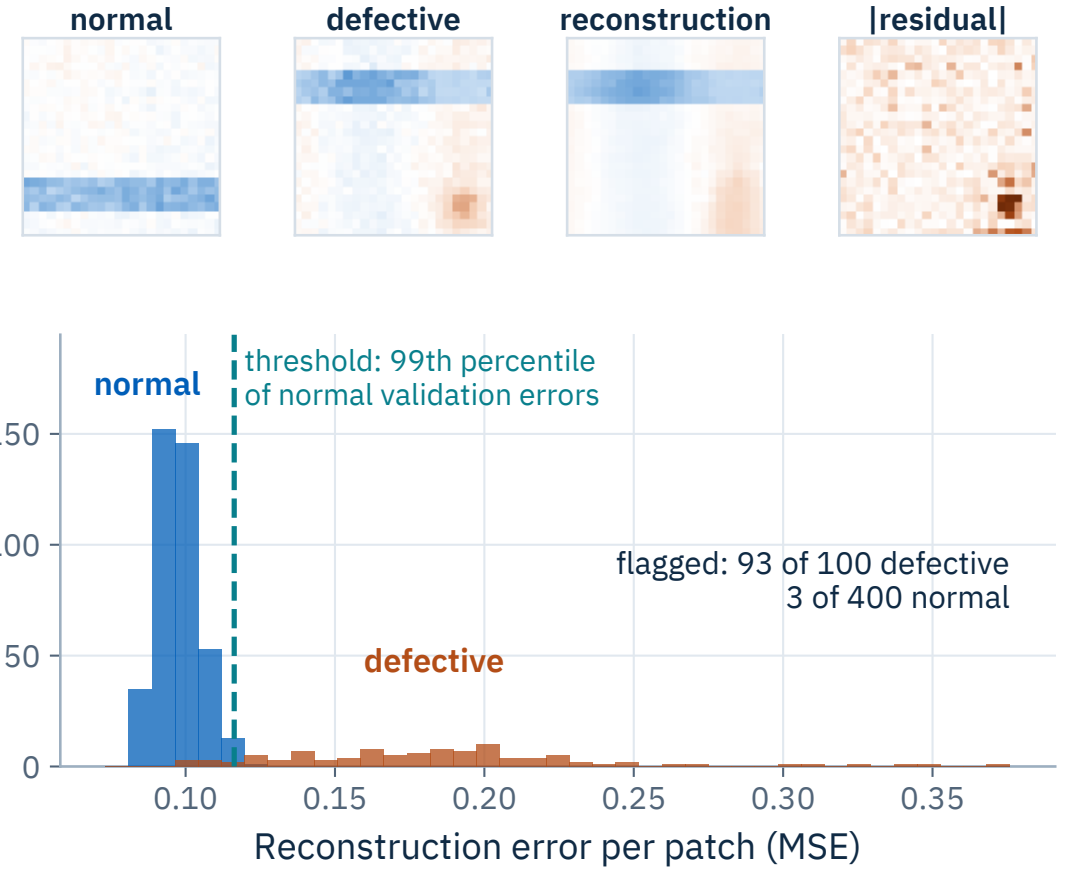
- Same normalization for input and target
- Store the statistics with the model

Match the output range

- Sigmoid only for targets in [0, 1]
- Standardized targets need a linear output

Reconstruction error can screen for anomalies

- Train on normal patches only; score each patch by its error.
- Set the threshold on normal validation data, never on test.
- The score is not a calibrated probability; strong models may rebuild defects too.



Synthetic casing patches (seed 11): 1,500 normal training, 400 normal validation, 400 normal and 100 defective test.

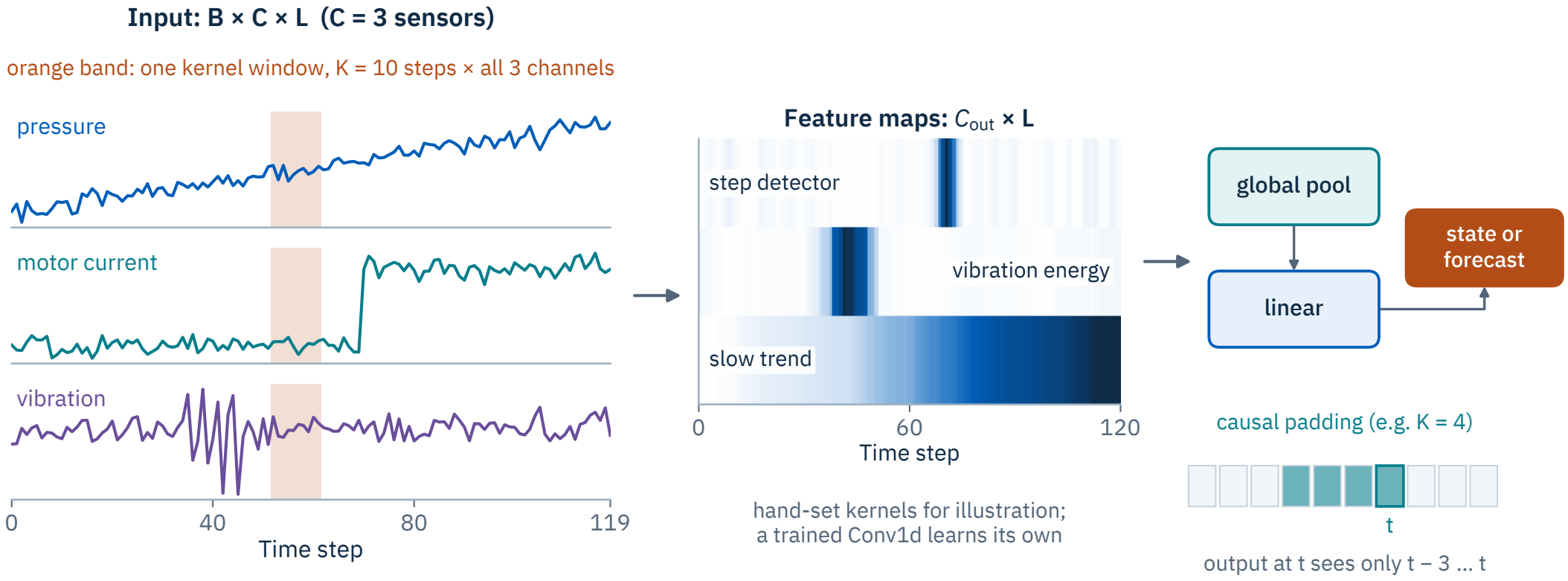
A 1-D convolution uses ordered neighbourhoods

Quantity	PyTorch shape	Example
Input	B × C × L	Sensor channels along time or depth
Kernel	C_{out} × C × K	Local window over the ordered axis
Feature map	B × C_{out} × L_{out}	Learned local signal patterns

Order and sampling interval must have a real meaning.

1-D CNNs read multichannel sensor windows

Kernels slide along time across all channels; causal padding hides the future (Day 5).



Synthetic three-sensor series (seed 21); hand-set kernels for illustration; schematic head and causal-padding inset.

Match the architecture to the data geometry and target

Core imagery

- Classify or segment visible patterns
- Split by well or core run

Well logs

- 1-D convolutions over depth
- Track receptive field in metres

Acoustic and ultrasonic inspection

- Acquisition-aware augmentation
- Respect azimuth wrap-around

Seismic

- Segment faults, horizons or bodies
- Test transfer across surveys and processing

◆ These are application directions, not demonstrated field results.

LABORATORY

55 min · Pairs

Laboratory 2A: a denoising convolutional autoencoder

- ☐ Check the encoder and decoder shapes with one batch.
- ☐ Train with noisy inputs and clean targets; plot training and validation loss.
- ☐ Compare latent sizes 8, 32 and 64 on the same validation images.
- ☐ Show noisy, reconstructed, clean and residual images for eight digits.

NOTEBOOK

`exercises/03_autoencoder/Conv_Autoencoder_Student.ipynb`

EVIDENCE TO SUBMIT

- Loss curves and a latent-size table
- A residual grid with one observation

⚠ Prepare: torchvision installed and MNIST downloaded to ./data. Choose 2A or 2B, not both.

LABORATORY

55 min · Pairs

Laboratory 2B: casing defects with grouped splits

- List runs, wells, missing values and flag fractions in the HDF5 file.
- Check the split by well (GroupShuffleSplit): wells, patches and prevalence per set.
- Train the CNN; choose the epoch and the threshold on validation wells.
- Report precision and recall per test well, once, at natural prevalence.
- Compare with a random patch split and explain the difference.

NOTEBOOK

`exercises/04_defect_detection/Defect_Detection_Student.ipynb`

EVIDENCE TO SUBMIT

- Split table: wells and patches per set
- Per-test-well metrics; random versus grouped scores

Prepare: install h5py; the HDF5 file is in the exercise folder. Its labels are named "prediction": check their provenance.

DISCUSSION

Lab 2 debrief: evidence before claims

Autoencoder path

- Did small latent codes blur digits or remove noise?
- Were residuals noise-like, or did strokes vanish?

vs

Casing-defect path

- How did metrics change between random and grouped splits?
- Which wells drive the errors, and why?

◆ Neither result is a field-performance claim: data, labels and splits define what the numbers mean.

Key takeaways

- ✓ Images are tensors whose axes have physical meaning; check shapes and spacing first.
- ✓ Convolution shares small local filters; padding, stride and dilation set the output size.
- ✓ Depth grows receptive fields; pooling and striding trade resolution for context.
- ✓ Residual paths, normalization and pretrained backbones make deep CNNs practical.
- ✓ Augmentations and splits must respect the physics and the data sources.
- ✓ Detection, segmentation and reconstruction change only the output.

PRACTICE

5 min · Individual

Exit check: explain the mechanism, not the name

1. Why is a convolution's parameter count independent of image width?
2. How does a stride of 2 change spatial resolution and receptive field growth?
3. Why can frozen weights still coexist with changing BatchNorm behaviour?
4. Name one augmentation you rejected today, and why.
5. What would convince you that a defect model is not using a shortcut?

References and further study

- LeCun, Bottou, Bengio, Haffner (1998). Gradient-based learning applied to document recognition. *Proc. IEEE* 86(11).
- He, Zhang, Ren, Sun (2016). Deep residual learning for image recognition. *CVPR*. Ioffe, Szegedy (2015). Batch normalization. *ICML*.
- Ronneberger, Fischer, Brox (2015). U-Net: convolutional networks for biomedical image segmentation. *MICCAI*.
- Dumoulin, Visin (2016). A guide to convolution arithmetic for deep learning. arXiv:1603.07285. Zhang (2019). Making convolutional networks shift-invariant again. *ICML*.
- Geirhos et al. (2020). Shortcut learning in deep neural networks. *Nature Machine Intelligence* 2. Goodfellow, Bengio, Courville (2016). *Deep Learning*, chapter 9.